Lab 5 Team Report: Healthcare REST API

Overview

This report follows the Lab 5 outline. Under each part, the four of us are shown one after another, always in the same order: Luca, Thanh, Zixuan, Gustave. The code is in the team repository: https://github.com/Thanh-Poth/Software_Engineering_Lab_4

Student	Route	Jira ticket	Pull request
Luca Perkins	/api/doctors	DEVOPS-10	#8
Thanh Potherat	/api/patients	DEVOPS-11	#5
Zixuan Qian	/api/appointments	DEVOPS-9	#6
Gustave Perdigeon	/api/metrics	DEVOPS-13	#9

2. JavaScript practice

The exercises are written in js-practice.js. Each one is shown below with its code.

2.1 Variables and data types

```
const hospitalName = 'Central Hospital'
const numberOfDoctors = 12
let isOpen = true

function welcome () {
  if (isOpen) {
    console.log(`Welcome to ${hospitalName}! We have ${numberOfDoctors} doctors currently available.`)
  } else {
    console.log(`${hospitalName} is currently closed. Please come back later.`)
  }
}
welcome()
isOpen = false
welcome()
```

Three variables, the welcome message, then the message when isOpen is false.

2.2 Arrays and loops

```
// 2.2 Arrays and loops
const doctors = ['Dr. Lee', 'Dr. Khan', 'Dr. Martin', 'Dr. Sarah', 'Dr. Amir']
for (const [i, doctor] of doctors.entries()) {
  console.log(`Doctor #${i + 1}: ${doctor}`)
}
doctors.push('Dr. Julie')
console.log(`Doctors count: ${doctors.length}`)

function findDoctor (name) {
  return doctors.includes(name) ? 'Doctor found' : 'Doctor not found'
}
console.log(findDoctor('Dr. Lee'))
console.log(findDoctor('Dr. House'))
```

Array of five doctors, for...of loop with the index, push() and findDoctor().

2.3 Objects and nested data

```
// 2.3 Objects and nested data
const patient = {
  name: 'Alice Martin',
  age: 34,
  conditions: ['diabetes', 'hypertension'],
  doctor: { name: 'Dr. Lee', specialty: 'Cardiology' }
}
console.log(patient.doctor.name)
console.log(patient.conditions.length)
console.log(`${patient.name} is treated by ${patient.doctor.name} (${patient.doctor.specialty}).`)
patient.conditions.push('anxiety')
console.log(patient)
```

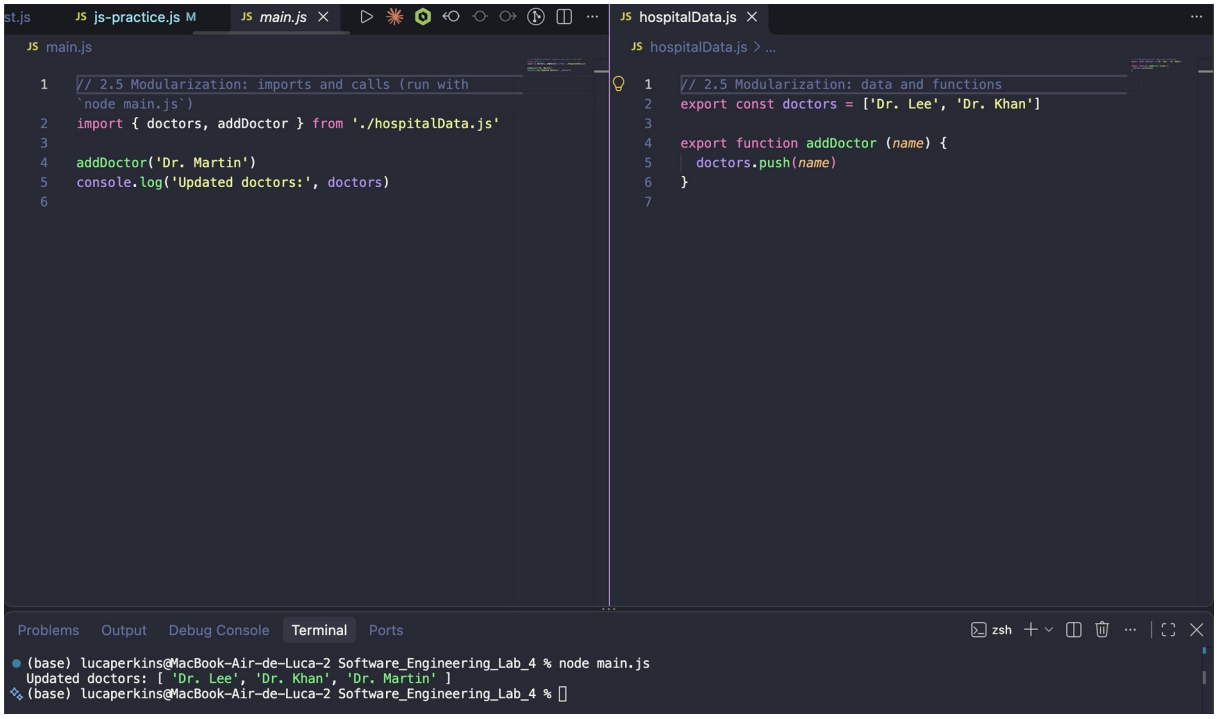
Nested patient object, the doctor name, the number of conditions and the new condition.

2.4 Functions and array filtering

```
// 2.4 Functions and array filtering
const patients = [
  { id: 1, name: 'Alice', age: 34 },
  { id: 2, name: 'John', age: 45 },
  { id: 3, name: 'Marie', age: 29 }
]
const filterByAge = (minAge) => patients.filter(p => p.age > minAge)
const addPatient = (name, age) => {
  patients.push({ id: patients.length + 1, name, age })
}
console.log('Older than 30:', filterByAge(30))
addPatient('Zoe', 51)
console.log('Updated patients:', patients)
```

filterByAge() and addPatient() on the patients array.

2.5 Modularization



main.js imports doctors and addDoctor from hospitalData.js. node main.js prints the updated list with Dr. Martin.

2.6 Error handling and validation

```
57  // 2.6 Error handling and validation
58  function createAppointment (doctor, patient, date) {
59    if (!doctor || !patient || !date) {
60      throw new Error('Missing required fields')
61    }
62    return { doctor, patient, date }
63  }
64  try {
65    console.log(createAppointment('Dr. Lee', 'Alice', '2025-03-15'))
66    console.log(createAppointment('Dr. Lee'))
67  } catch (err) {
68    console.error('Error:', err.message)
69  }
```

createAppointment() throws an error when a field is missing, caught with try/catch.

2.7 Asynchronous code

```
71  // 2.7 Asynchronous code
72  function getHospitalData () {
73    return new Promise((resolve) => {
74      setTimeout(() => resolve({ name: 'Central Hospital', uptime: 1234 }), 1000)
75    })
76  }
77  async function showData () {
78    console.log('Fetching hospital data...')
79    const data = await getHospitalData()
80    console.log('Data loaded:', data)
81  }
82  showData()
```

A Promise resolved after 1000 ms and read with await.

Running js-practice.js

```
(base) lucaperkins@MacBook-Air-de-Luca-2 Software_Engineering_Lab_4 % node js-practice.js
Welcome to Central Hospital! We have 12 doctors currently available.
Central Hospital is currently closed. Please come back later.
Doctor #1: Dr. Lee
Doctor #2: Dr. Khan
Doctor #3: Dr. Martin
Doctor #4: Dr. Sarah
Doctor #5: Dr. Amir
Doctors count: 6
Doctor found
Doctor not found
Dr. Lee
2
Alice Martin is treated by Dr. Lee (Cardiology).
{
  name: 'Alice Martin',
  age: 34,
  conditions: [ 'diabetes', 'hypertension', 'anxiety' ],
  doctor: { name: 'Dr. Lee', specialty: 'Cardiology' }
}
Older than 30: [ { id: 1, name: 'Alice', age: 34 }, { id: 2, name: 'John', age: 45 } ]
Updated patients: [
  { id: 1, name: 'Alice', age: 34 },
  { id: 2, name: 'John', age: 45 },
  { id: 3, name: 'Marie', age: 29 },
  { id: 4, name: 'Zoe', age: 51 }
]
{ doctor: 'Dr. Lee', patient: 'Alice', date: '2025-03-15' }
Error: Missing required fields
Fetching hospital data...
Data loaded: { name: 'Central Hospital', uptime: 1234 }
```

Output of node js-practice.js, all sections run without error.

3. Create your first APIs

Each of us implemented one route and tested it with Vitest and Supertest. Files in src/routes/auto are mounted automatically.

3.1 The routes

```
/**
 * Doctors API.
 * GET  /api/doctors → list all doctors
 * POST /api/doctors → create a doctor { name, specialty }
 */
import express from 'express'
const router = express.Router()

const doctors = [
  { id: 1, name: 'Dr. Sarah Lee', specialty: 'Cardiology' },
  { id: 2, name: 'Dr. Amir Khan', specialty: 'Pediatrics' }
]

router.get('/api/doctors', (_req, res) => res.status(200).json(doctors))

router.post('/api/doctors', (req, res) => {
  const { name, specialty } = req.body ?? {}
  if (typeof name !== 'string' || !name.trim() || typeof specialty !== 'string' || !specialty.trim()) {
    return res.status(400).json({ error: 'Missing or invalid fields (name, specialty: non-empty strings)' })
  }
  const newDoctor = { id: doctors.length + 1, name: name.trim(), specialty: specialty.trim() }
  doctors.push(newDoctor)
  return res.status(201).json(newDoctor)
})

export default router

```

Luca Perkins: doctors route, GET and POST /api/doctors.

```
/**
 * Patients API.
 * GET  /api/patients → list all patients
 * POST /api/patients → create a patient { name, age }
 */
import { Router } from 'express'

const router = Router()

const patients = [
  { id: 1, name: 'Alice', age: 30 },
  { id: 2, name: 'John', age: 45 }
]

router.get('/api/patients', (_req, res) => res.status(200).json(patients))

router.post('/api/patients', (req, res) => {
  const { name, age } = req.body
  if (typeof name !== 'string' || !name.trim() || !Number.isInteger(age) || age <= 0) {
    return res.status(400).json({ error: 'Missing or invalid fields (name: string, age: positive integer)' })
  }
  const newPatient = { id: patients.length + 1, name: name.trim(), age }
  patients.push(newPatient)
  res.status(201).json(newPatient)
})

export default router

```

Thanh Potherat: patients route, GET and POST /api/patients.

```js
import { Router } from 'express'

const router = Router()

// In-memory storage: reset when the server restarts.
const appointments = []
let nextId = 1

// Retrieve all appointments.
router.get('/api/appointments', (_req, res) => {
  res.status(200).json(appointments)
})

// Create an appointment.
router.post('/api/appointments', (req, res) => {
  const { doctor, patient, date } = req.body || {}

  // Each required field must be a non-empty string.
  const fields = [doctor, patient, date]
  const valid = fields.every(
    value => typeof value === 'string' && value.trim().length > 0
  )

  if (!valid) {
    return res.status(400).json({
      error: 'Doctor, patient and date are required non-empty strings'
    })
  }

  const appointment = {
    id: nextId++,
    doctor: doctor.trim(),
    patient: patient.trim(),
    date: date.trim()
  }

  appointments.push(appointment)

  return res.status(201).json(appointment)
})

export default router

```

Zixuan Qian: appointments route, GET and POST /api/appointments.

```js
import { Router } from 'express'
import { patients } from './patients.route.js'

const router = Router()

router.get('/api/metrics', (_req, res) => {
  res.status(200).json({
    uptimeSeconds: process.uptime(),
    memoryUsedMB: Math.round(
      process.memoryUsage().rss / (1024 * 1024) * 100
    ) / 100,
    totalPatients: patients.length
  })
})

export default router

```

Gustave Perdigeon: metrics route, GET /api/metrics.

3.2 The tests

```
/**
 * Integration tests for /api/doctors.
 */
import request from 'supertest'
import app from '../src/app.js'
import { describe, it, expect } from 'vitest'

describe('GET /api/doctors', () => {
  it('returns an array of doctors', async () => {
    const res = await request(app).get('/api/doctors')
    expect(res.status).toBe(200)
    expect(Array.isArray(res.body)).toBe(true)
    expect(res.body[0]).toEqual({ id: 1, name: 'Dr. Sarah Lee', specialty: 'Cardiology' })
  })
})

describe('POST /api/doctors', () => {
  it('creates a doctor and returns 201 with an id', async () => {
    const res = await request(app).post('/api/doctors').send({ name: 'Dr. Julie', specialty: 'Neurology' })
    expect(res.status).toBe(201)
    expect(res.body).toHaveProperty('id')
    expect(res.body).toMatchObject({ name: 'Dr. Julie', specialty: 'Neurology' })
  })

  it('makes the new doctor visible in GET', async () => {
    await request(app).post('/api/doctors').send({ name: 'Dr. Paul', specialty: 'ENT' })
    const res = await request(app).get('/api/doctors')
    expect(res.body.some(d => d.name === 'Dr. Paul')).toBe(true)
  })

  it.each([
    [{}],
    [{ name: 'Dr. NoSpecialty' }],
    [{ specialty: 'Cardiology' }],
    [{ name: '  ', specialty: 'ENT' }],
    [{ name: 'Dr. Bad', specialty: 42 }]
  ])('returns 400 for invalid body %j', async (body) => {
    const res = await request(app).post('/api/doctors').send(body)
    expect(res.status).toBe(400)
    expect(res.body).toHaveProperty('error')
  })

  it('returns 400 when no body is sent', async () => {
```

Luca Perkins: doctors.test.js, GET and POST tests.

```
describe('POST /api/doctors', () => {

  it.each([
    [{}],
    [{ name: 'Dr. NoSpecialty' }],
    [{ specialty: 'Cardiology' }],
    [{ name: '  ', specialty: 'ENT' }],
    [{ name: 'Dr. Bad', specialty: 42 }]
  ])('returns 400 for invalid body %j', async (body) => {
    const res = await request(app).post('/api/doctors').send(body)
    expect(res.status).toBe(400)
    expect(res.body).toHaveProperty('error')
  })

  it('returns 400 when no body is sent', async () => {
    const res = await request(app).post('/api/doctors')
    expect(res.status).toBe(400)
  })
})

```

Luca Perkins: doctors.test.js, invalid bodies and missing body return 400.

```
/**
 * Integration tests for /api/patients.
 */
import request from 'supertest'
import app from '../src/app.js'
import { describe, it, expect } from 'vitest'

describe('GET /api/patients', () => {
  it('returns an array of patients', async () => {
    const res = await request(app).get('/api/patients')
    expect(res.status).toBe(200)
    expect(Array.isArray(res.body)).toBe(true)
    expect(res.body[0]).toHaveProperty('name')
  })
})

describe('POST /api/patients', () => {
  it('creates a patient and returns 201 with an id', async () => {
    const res = await request(app).post('/api/patients').send({ name: 'Marie', age: 29 })
    expect(res.status).toBe(201)
    expect(res.body).toHaveProperty('id')
    expect(res.body).toMatchObject({ name: 'Marie', age: 29 })
  })

  it('makes the new patient visible in GET', async () => {
    await request(app).post('/api/patients').send({ name: 'Zoe', age: 51 })
    const res = await request(app).get('/api/patients')
    expect(res.body.some(p => p.name === 'Zoe')).toBe(true)
  })

  it.each([
    [{}],
    [{ name: 'NoAge' }],
    [{ age: 20 }],
    [{ name: '  ', age: 20 }],
    [{ name: 'Bad', age: -3 }],
    [{ name: 'Bad', age: 'old' }]
  ])('returns 400 for invalid body %j', async (body) => {
    const res = await request(app).post('/api/patients').send(body)
    expect(res.status).toBe(400)
    expect(res.body).toHaveProperty('error')
  })

```

Thanh Potherat: patients.test.js, GET and POST tests.

```
describe('POST /api/patients', () => {
    [{ name: 'NoAge' }],
    [{ age: 20 }],
    [{ name: '  ', age: 20 }],
    [{ name: 'Bad', age: -3 }],
    [{ name: 'Bad', age: 'old' }]
  ])('returns 400 for invalid body %j', async (body) => {
    const res = await request(app).post('/api/patients').send(body)
    expect(res.status).toBe(400)
    expect(res.body).toHaveProperty('error')
  })

  it('returns 400 when no body is sent', async () => {
    const res = await request(app).post('/api/patients')
    expect(res.status).toBe(400)
  })
})

```

Thanh Potherat: patients.test.js, invalid bodies and missing body return 400.

```
import request from 'supertest'
import app from '../src/app.js'
import { describe, it, expect } from 'vitest'

const endpoint = '/api/appointments'

const validAppointment = {
  doctor: 'Dr. Lee',
  patient: 'Alice Martin',
  date: '2026-12-10T10:00:00Z'
}

describe('Appointments API', () => {
  it('returns an array with HTTP 200', async () => {
    const res = await request(app).get(endpoint)

    expect(res.status).toBe(200)
    expect(res.headers['content-type']).toMatch(/application\/json/)
    expect(Array.isArray(res.body)).toBe(true)
  })

  it('creates an appointment and makes it available in the list', async () => {
    const res = await request(app)
      .post(endpoint)
      .send(validAppointment)

    expect(res.status).toBe(201)
    expect(res.body).toEqual({
      id: expect.any(Number),
      ...validAppointment
    })

    const list = await request(app).get(endpoint)

    expect(list.status).toBe(200)
    expect(list.body).toContainEqual(res.body)
  })

  it('assigns distinct IDs to successive appointments', async () => {
    const first = await request(app).post(endpoint).send(validAppointment)
    const second = await request(app).post(endpoint).send(validAppointment)

    expect(first.status).toBe(201)
```

Zixuan Qian: appointments.test.js, GET, creation and distinct ids.

```
13  describe('Appointments API', () => {
39    it('assigns distinct IDs to successive appointments', async () => {
46    })
47
48    it('removes surrounding spaces from accepted fields', async () => {
49      const res = await request(app).post(endpoint).send({
50        doctor: '  Dr. Lee  ',
51        patient: '  Alice Martin  ',
52        date: '  2026-12-10T10:00:00Z  '
53      })
54
55      expect(res.status).toBe(201)
56      expect(res.body).toEqual({
57        id: expect.any(Number),
58        ...validAppointment
59      })
60    })
61
62    for (const field of ['doctor', 'patient', 'date']) {
63      for (const invalidValue of [undefined, '', '   ', null, 123]) {
64        it(`rejects ${field}=${String(invalidValue)} without adding an appointment`, async () => {
65          const before = await request(app).get(endpoint)
66          const payload = { ...validAppointment }
67
68          if (invalidValue === undefined) {
69            delete payload[field]
70          } else {
71            payload[field] = invalidValue
72          }
73
74          const res = await request(app).post(endpoint).send(payload)
75
76          expect(res.status).toBe(400)
77          expect(res.body).toHaveProperty('error', expect.any(String))
78
79          const after = await request(app).get(endpoint)
80
81          expect(after.body).toEqual(before.body)
82        })
83      }
84    }
85
86    it('rejects a request without a body', async () => {
87      const before = await request(app).get(endpoint)
```

Zixuan Qian: appointments.test.js, trimmed spaces and rejected fields.

```
describe('Appointments API', () => {
      it(`rejects ${field}=${String(invalidValue)} without adding an appointment`, async () => {

        expect(res.status).toBe(400)
        expect(res.body).toHaveProperty('error', expect.any(String))

        const after = await request(app).get(endpoint)

        expect(after.body).toEqual(before.body)
      })
    }
  }

  it('rejects a request without a body', async () => {
    const before = await request(app).get(endpoint)
    const res = await request(app).post(endpoint)

    expect(res.status).toBe(400)

    const after = await request(app).get(endpoint)

    expect(after.body).toEqual(before.body)
  })
})
```

Zixuan Qian: appointments.test.js, request without a body.

```
import request from 'supertest'
import { describe, it, expect } from 'vitest'
import app from '../src/app.js'

describe('GET /api/metrics', () => {
  it('returns system statistics as JSON', async () => {
    const res = await request(app).get('/api/metrics')

    expect(res.status).toBe(200)
    expect(res.headers['content-type']).toContain('application/json')
    expect(res.body.uptimeSeconds).toEqual(expect.any(Number))
    expect(res.body.uptimeSeconds).toBeGreaterThanOrEqual(0)
    expect(res.body.memoryUsedMB).toEqual(expect.any(Number))
    expect(res.body.memoryUsedMB).toBeGreaterThan(0)
  })

  it('returns the current number of patients', async () => {
    const patients = await request(app).get('/api/patients')
    const metrics = await request(app).get('/api/metrics')

    expect(metrics.status).toBe(200)
    expect(metrics.body.totalPatients).toBe(patients.body.length)
  })

  it('updates the count after adding a patient', async () => {
    const before = await request(app).get('/api/metrics')

    const created = await request(app)
      .post('/api/patients')
      .send({ name: 'Metrics Test', age: 30 })

    expect(created.status).toBe(201)

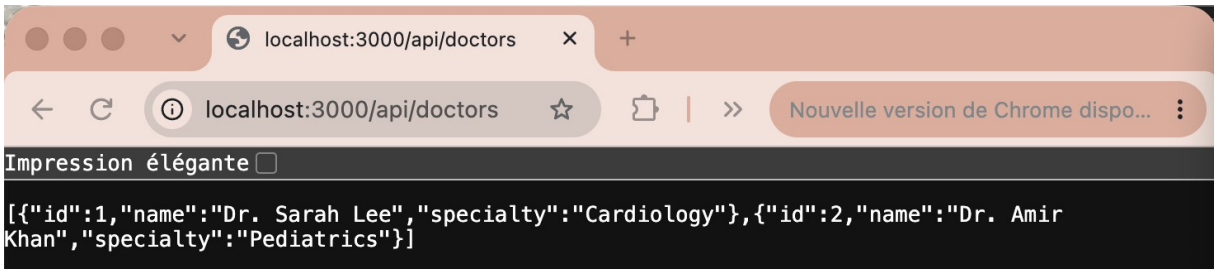
    const after = await request(app).get('/api/metrics')

    expect(after.body.totalPatients).toBe(before.body.totalPatients + 1)
  })
})
```

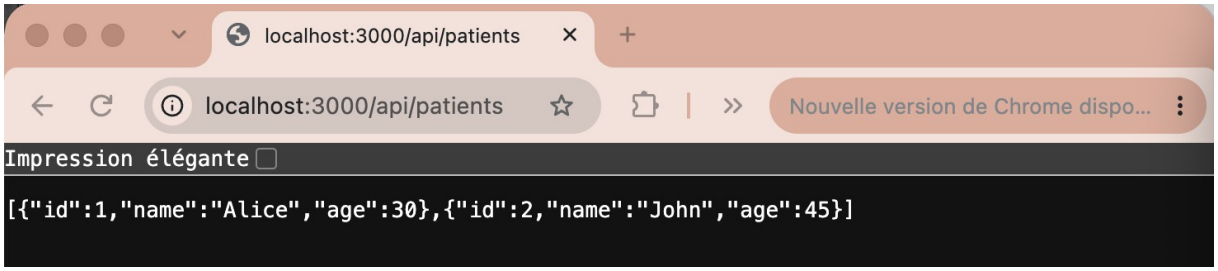
Gustave Perdigeon: metrics.test.js, statistics as JSON, patient count, and the count after a POST.

3.3 Local checks

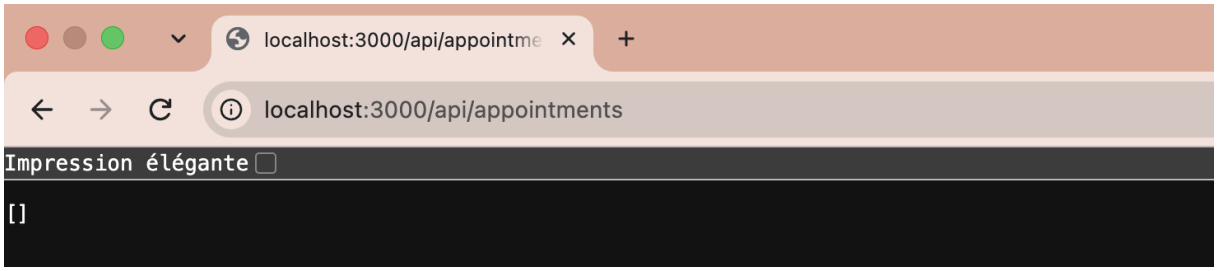
With the server started (npm run dev), each route answers in the browser.



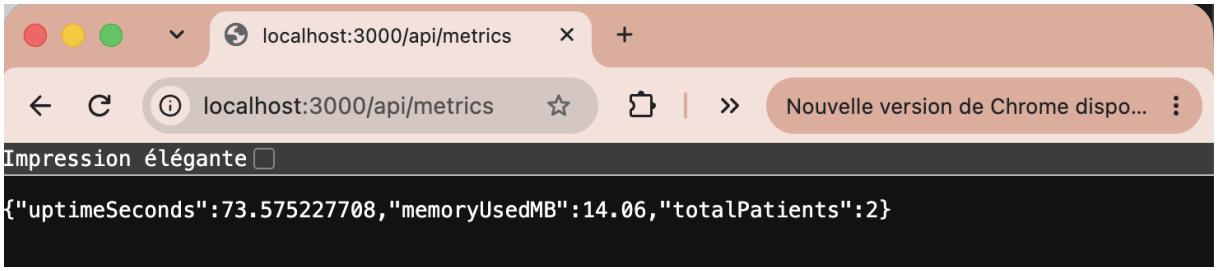
Luca Perkins: /api/doctors returns the two doctors.



Thanh Potherat: /api/patients returns the two patients.



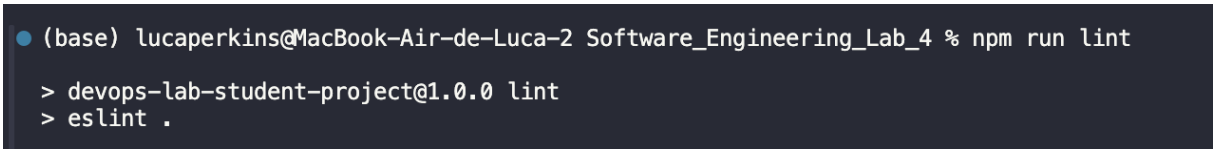
Zixuan Qian: /api/appointments returns an empty list.



Gustave Perdigeon: /api/metrics returns uptime, memory and totalPatients.



npm test on main: 11 test files and 57 tests passed.

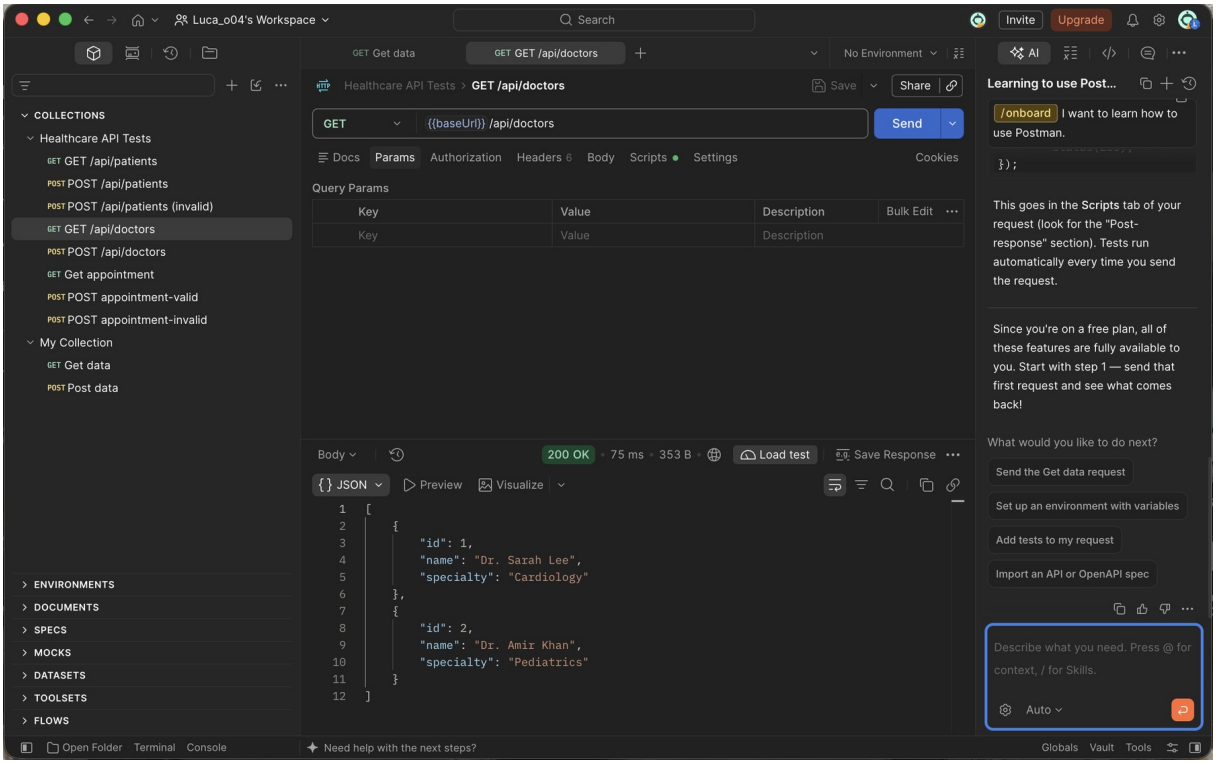


npm run lint finishes without any error.

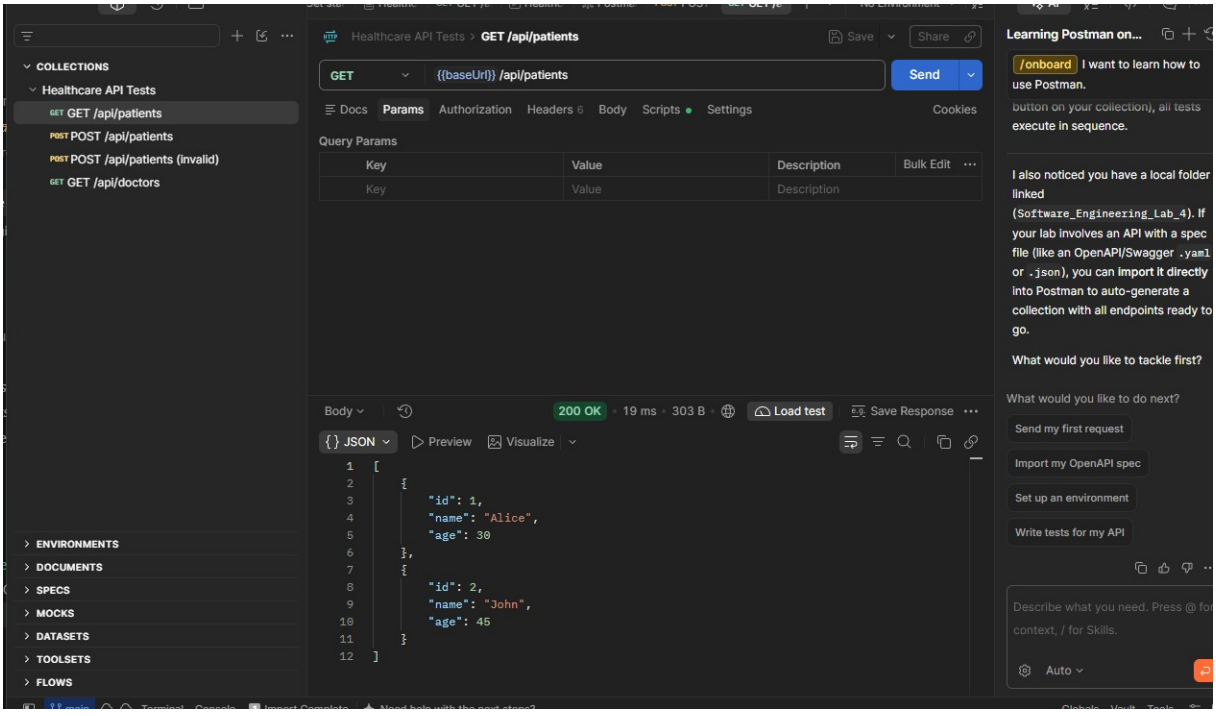
4. Test and validate your API with Postman

Requests were sent from the Postman desktop app to the local server (npm run dev, http://localhost:3000).

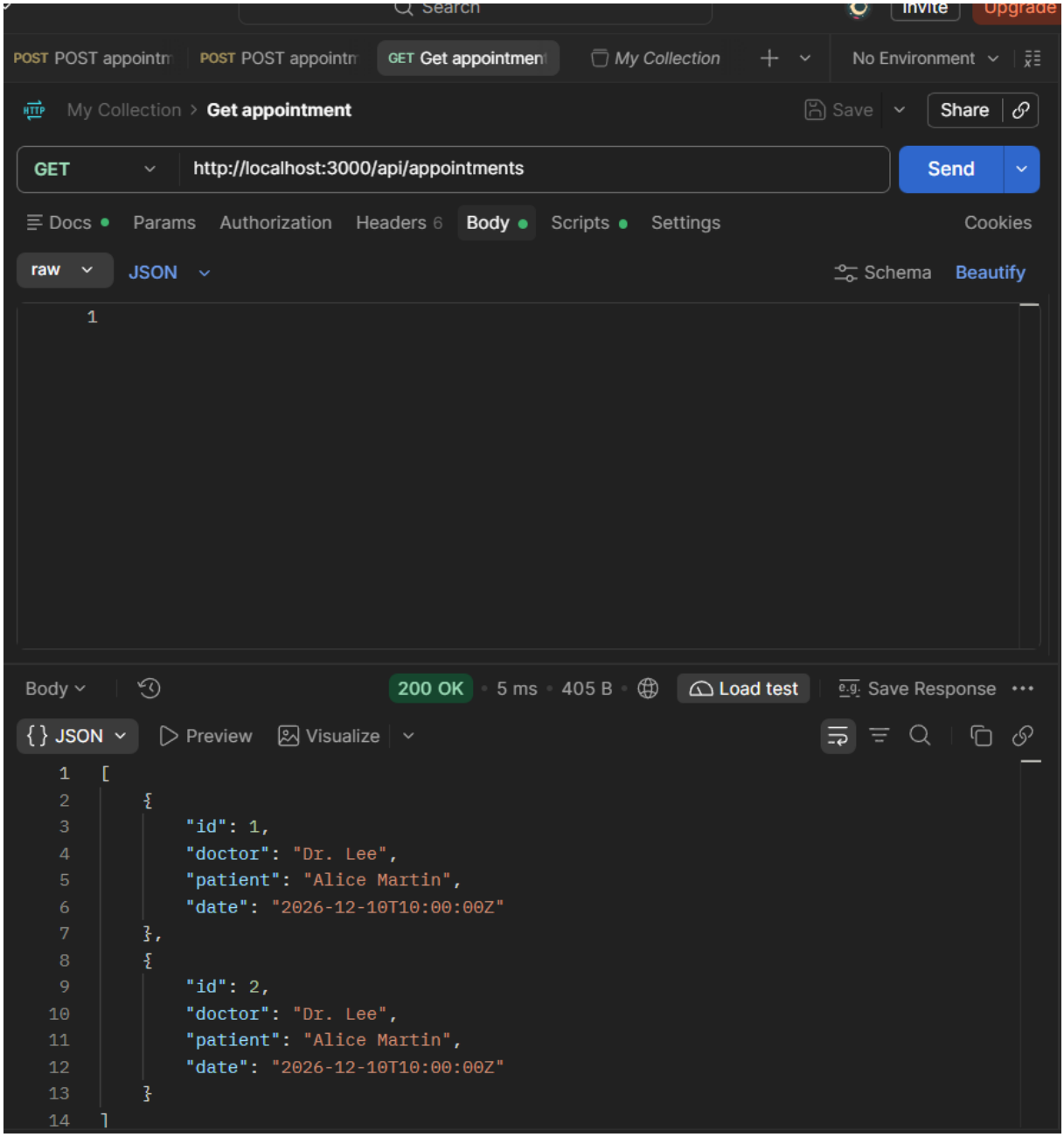
4.3 GET requests



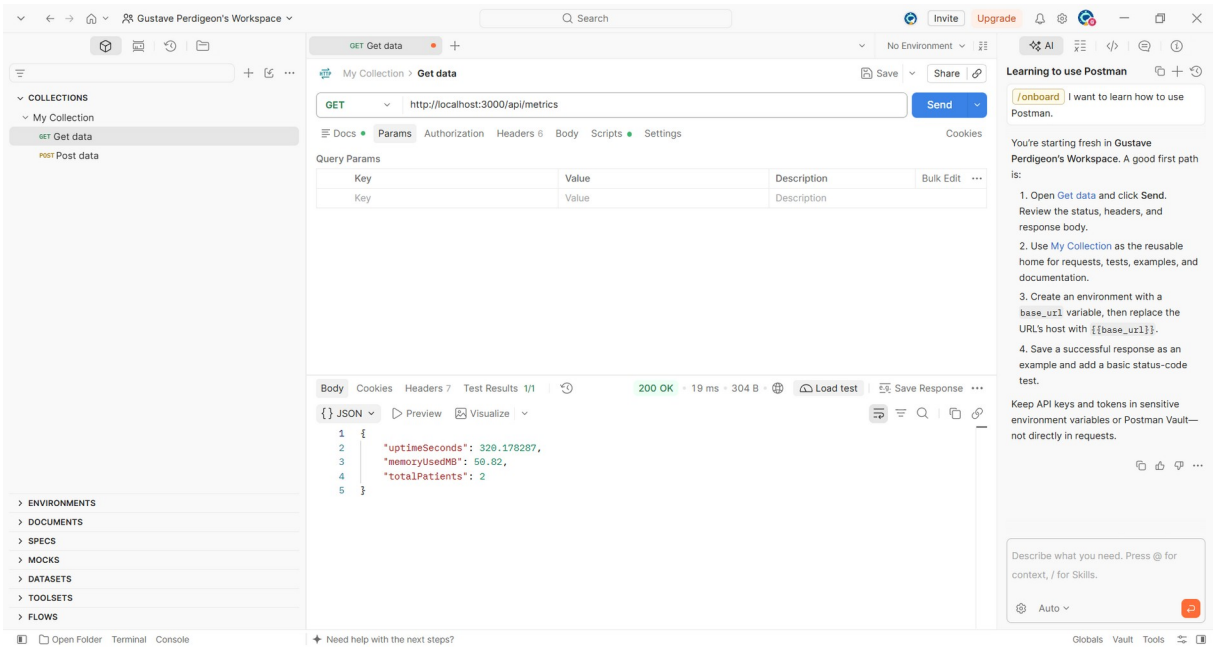
Luca Perkins: GET /api/doctors returns 200 OK with the two doctors.



Thanh Potherat: GET /api/patients returns 200 OK with the two patients.

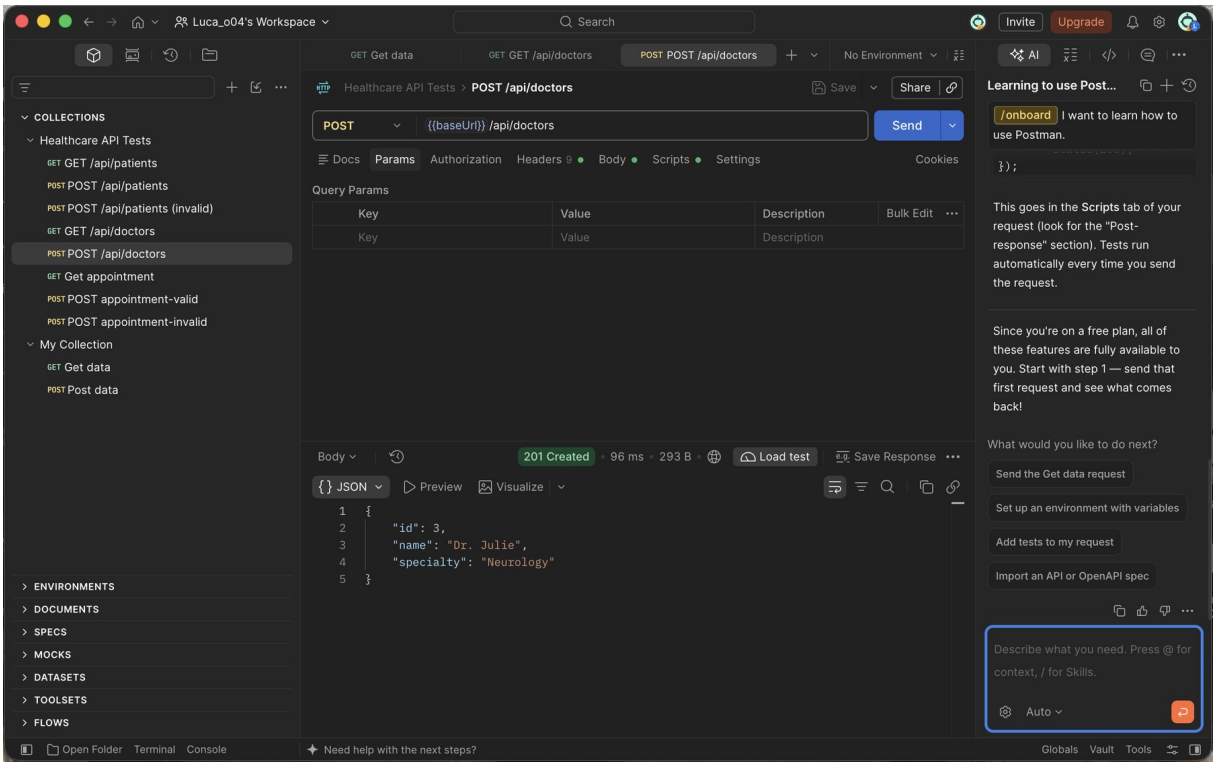


Zixuan Qian: GET /api/appointments returns 200 OK.

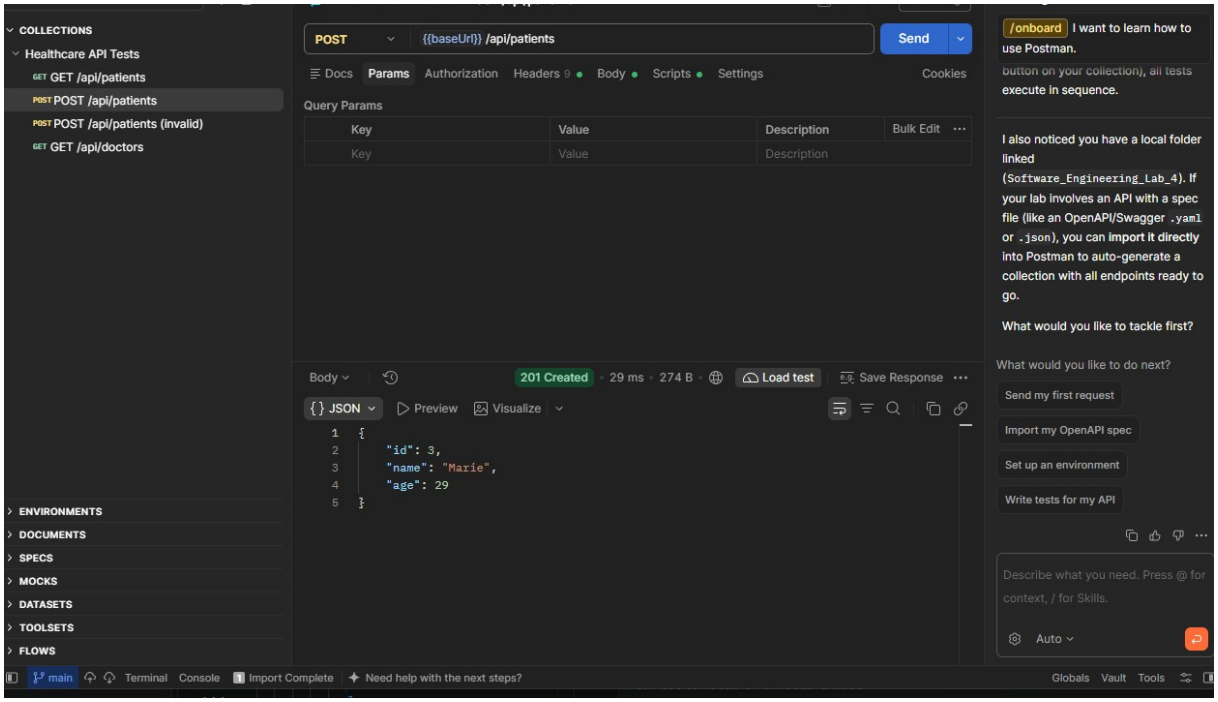


Gustave Perdigeon: GET /api/metrics returns 200 OK with the three statistics.

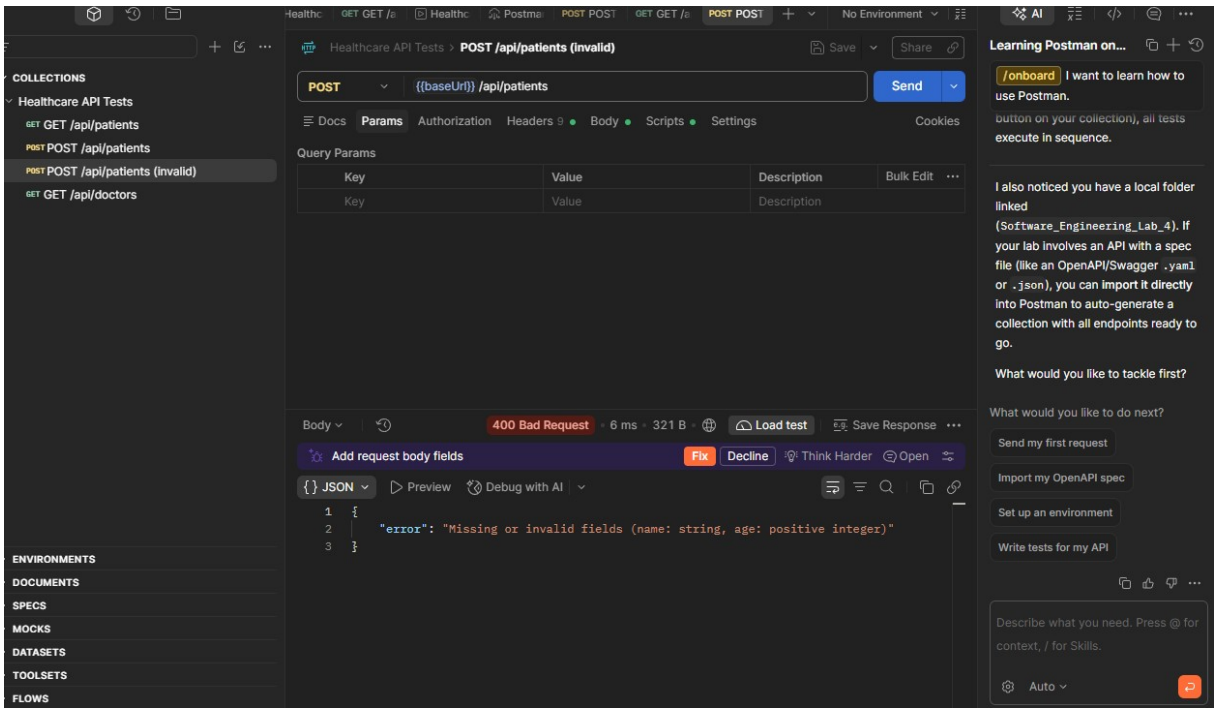
4.4 POST requests



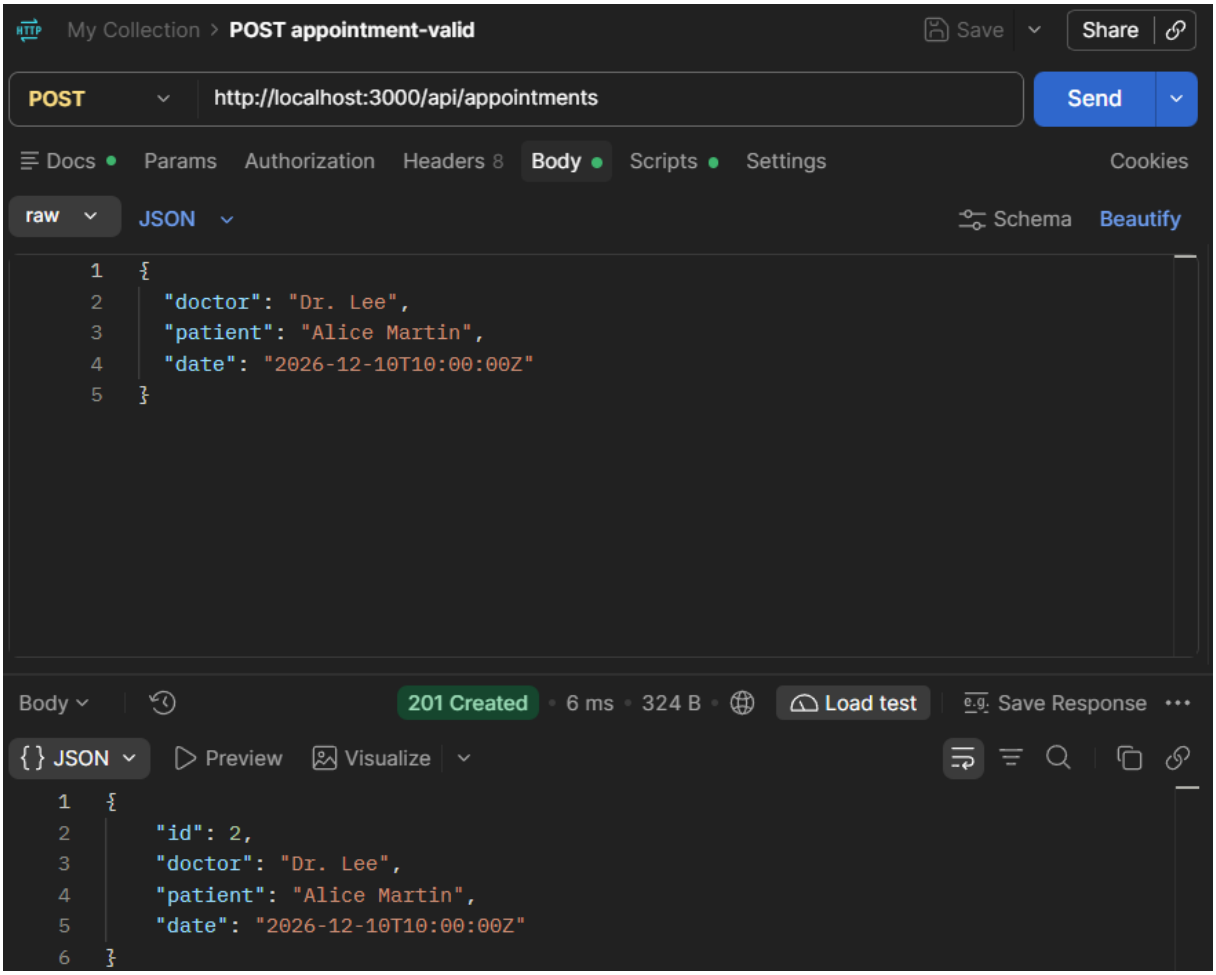
Luca Perkins: POST /api/doctors returns 201 Created with the new doctor, id 3.



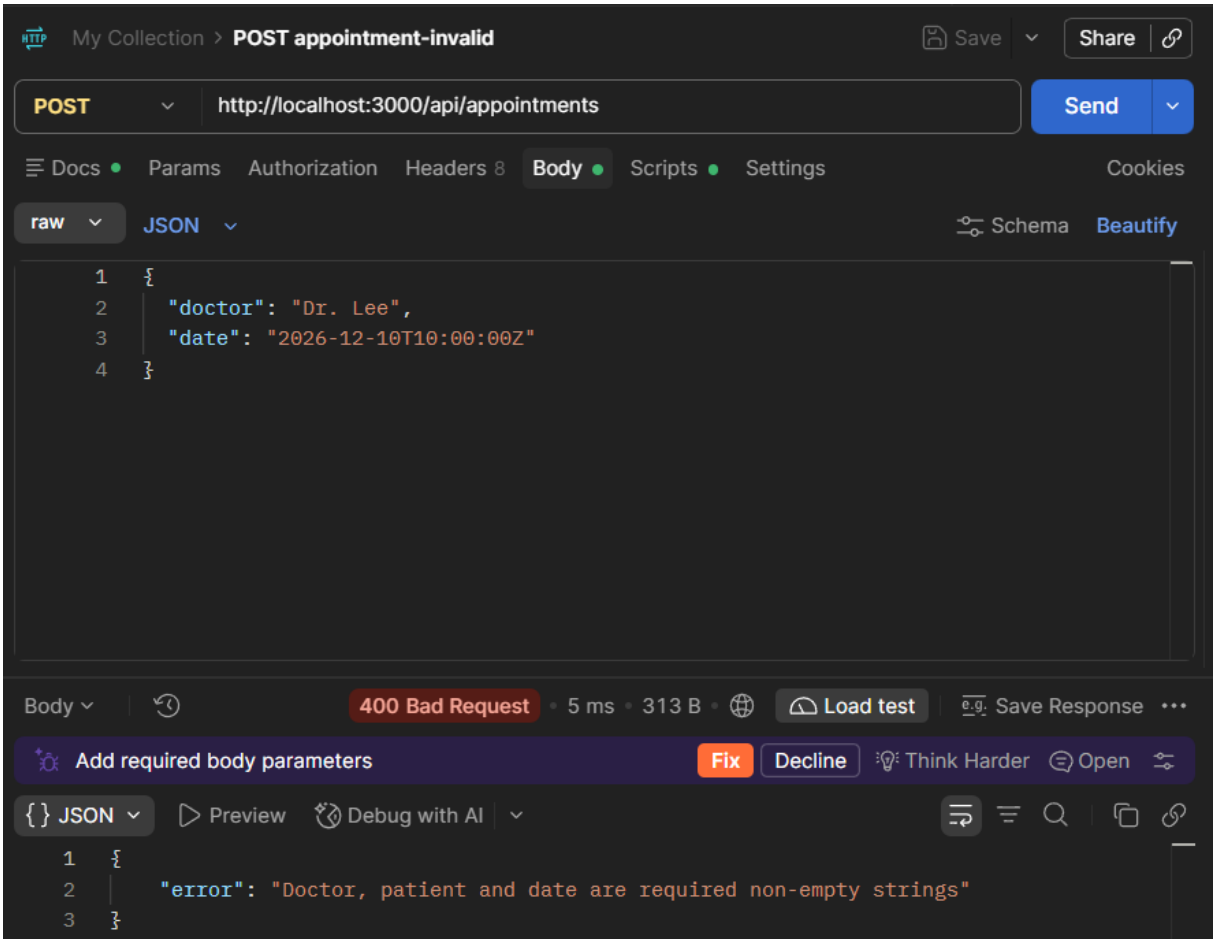
Thanh Potherat: POST /api/patients returns 201 Created, patient Marie with id 3.



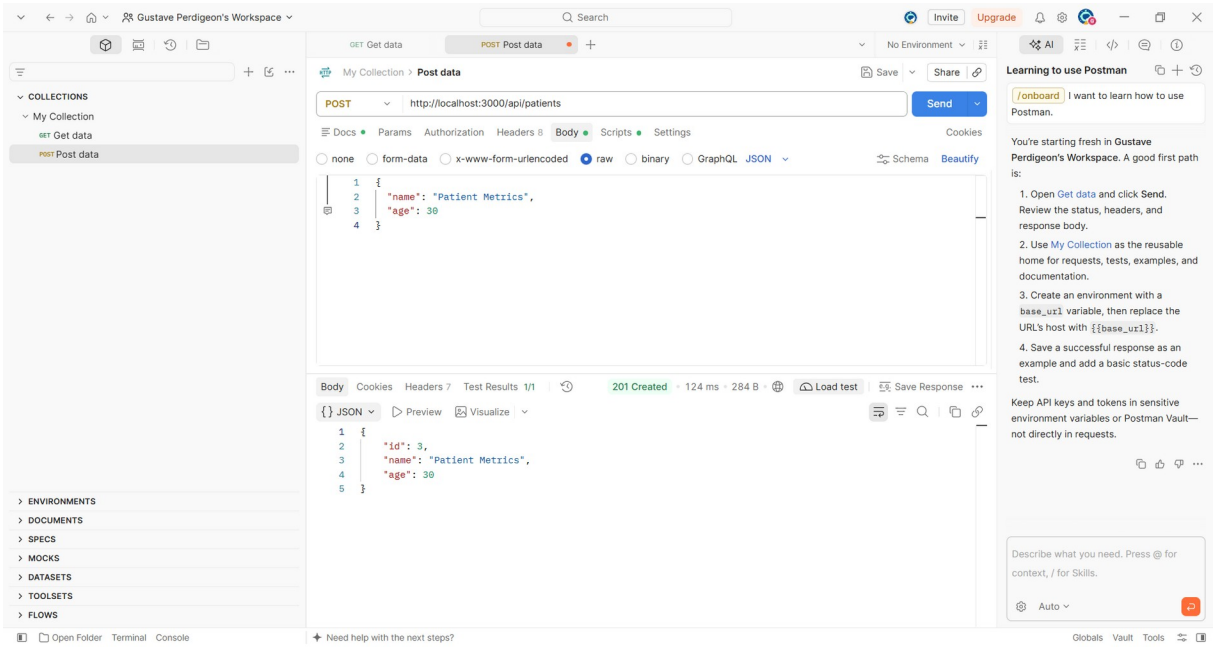
Thanh Potherat: POST /api/patients without an age returns 400 Bad Request.



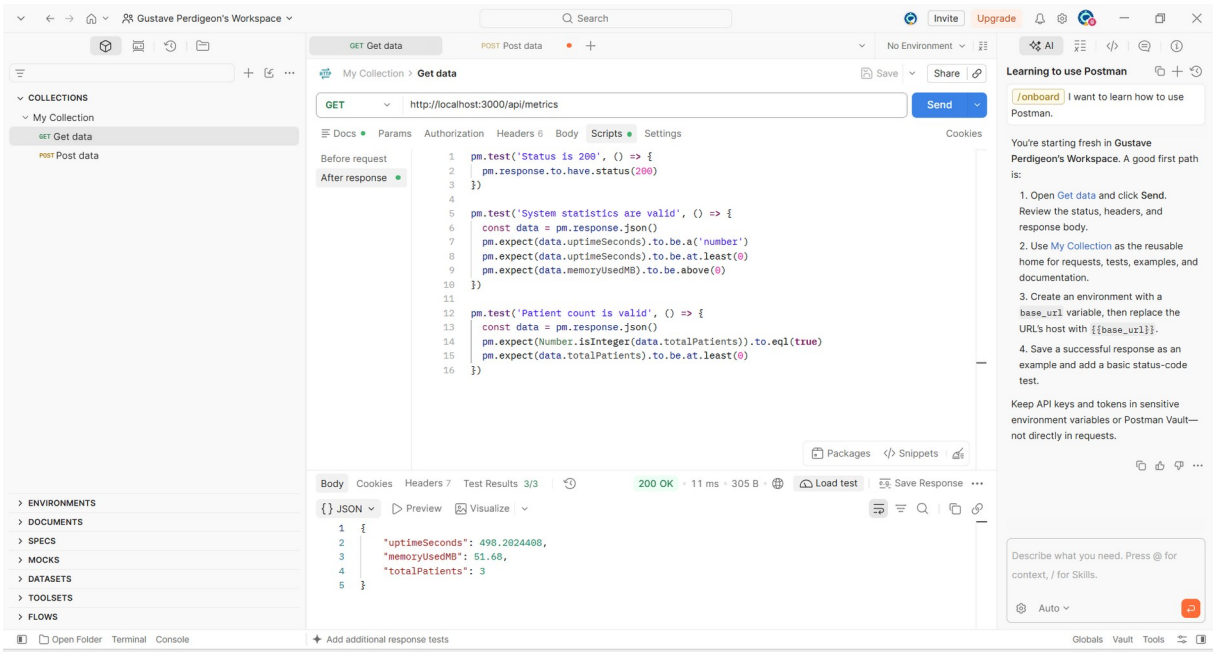
Zixuan Qian: POST /api/appointments with a valid body returns 201 Created.



Zixuan Qian: the same POST without a patient returns 400 Bad Request.

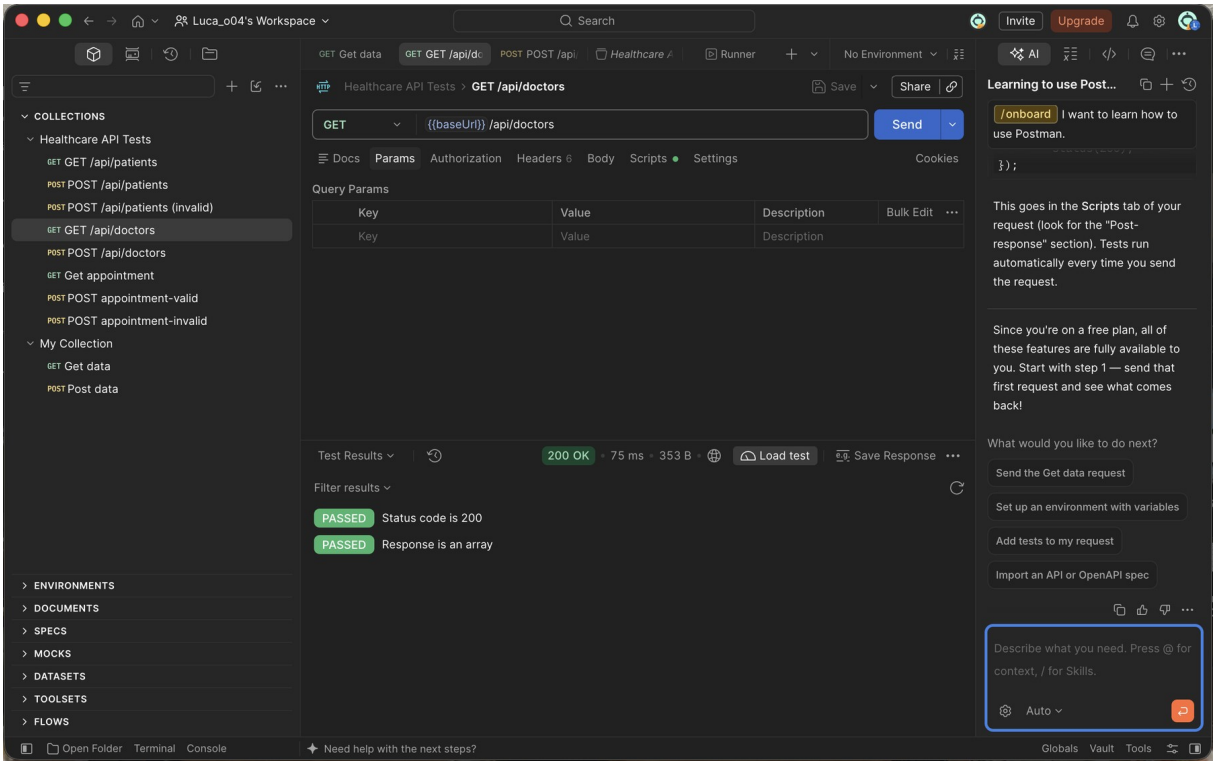


Gustave Perdigeon: POST /api/patients returns 201 Created, patient with id 3.

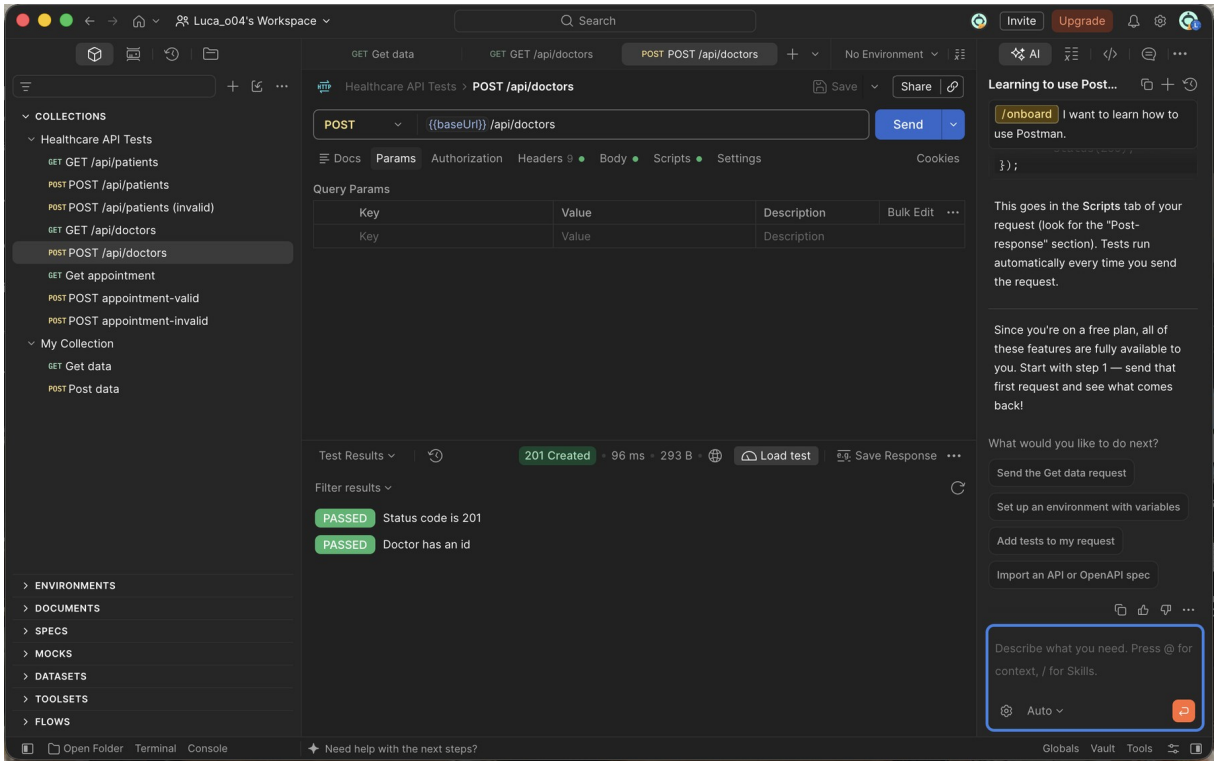


Gustave Perdigeon: GET /api/metrics again, totalPatients went from 2 to 3.

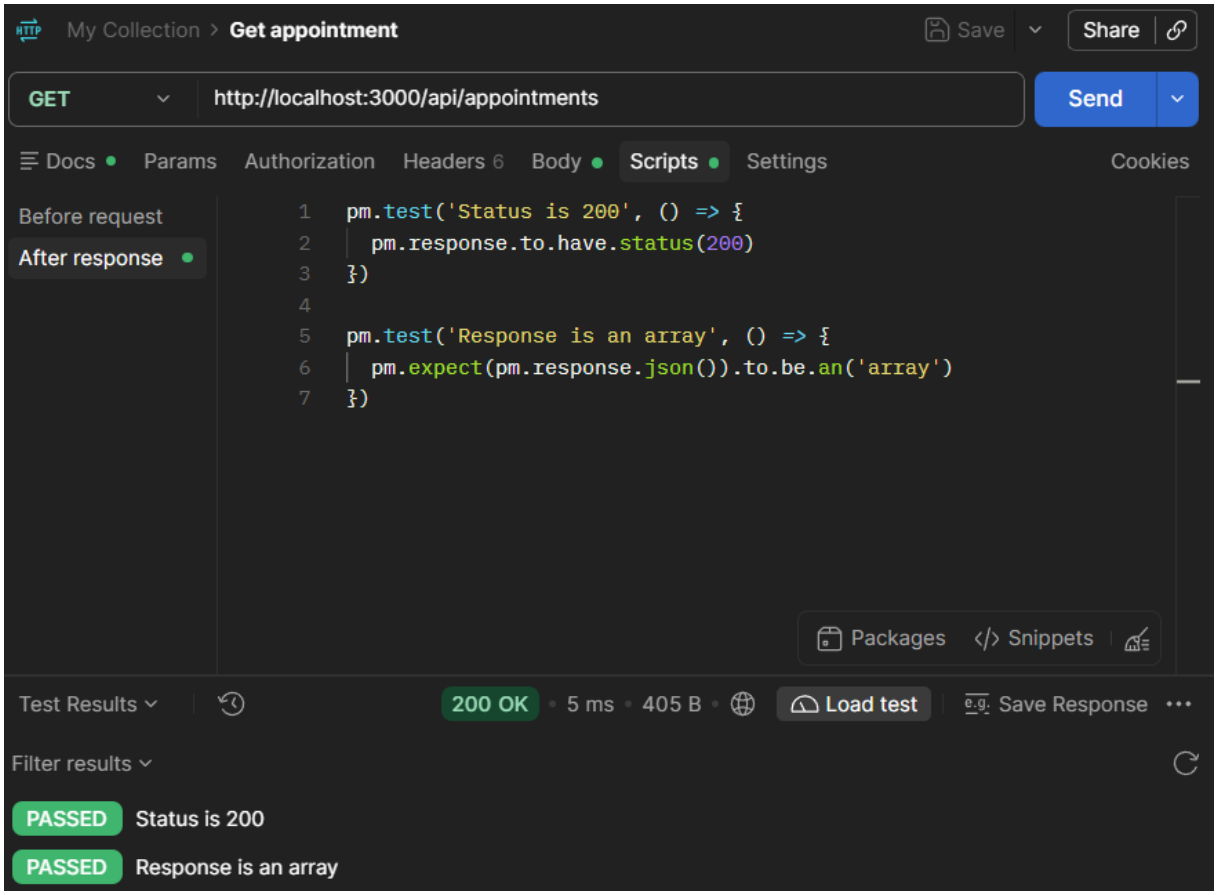
4.5 Automating the tests



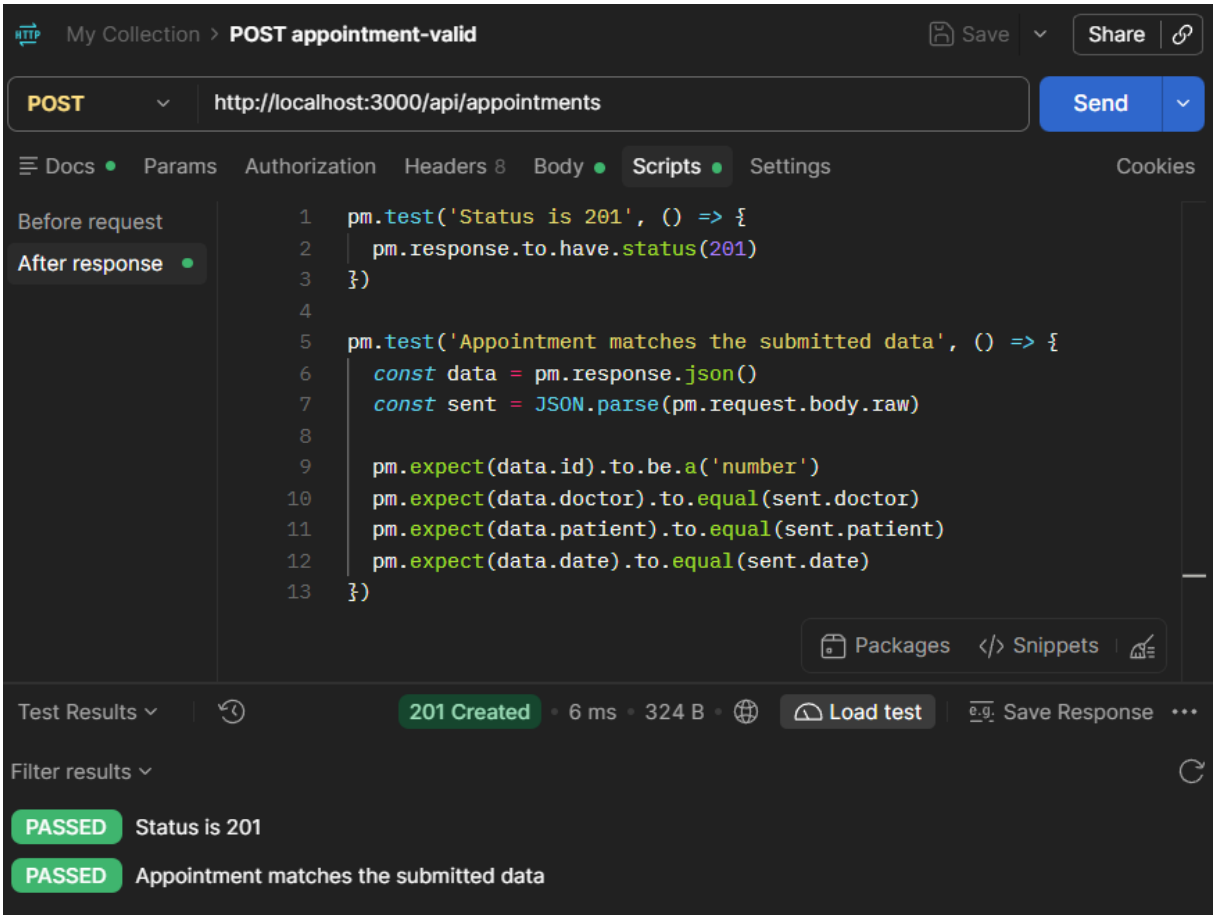
Luca Perkins: GET /api/doctors, both assertions passed.



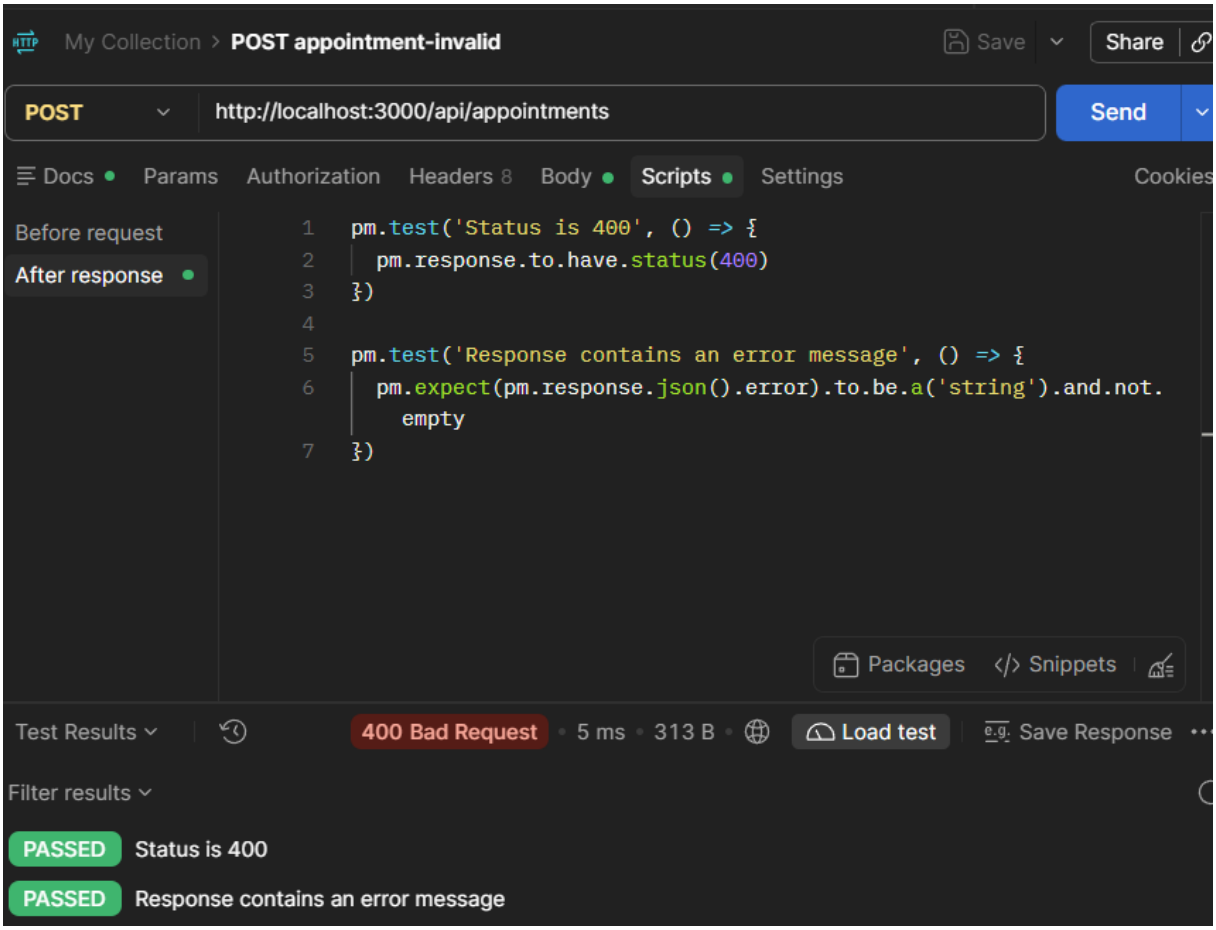
Luca Perkins: POST /api/doctors, both assertions passed.



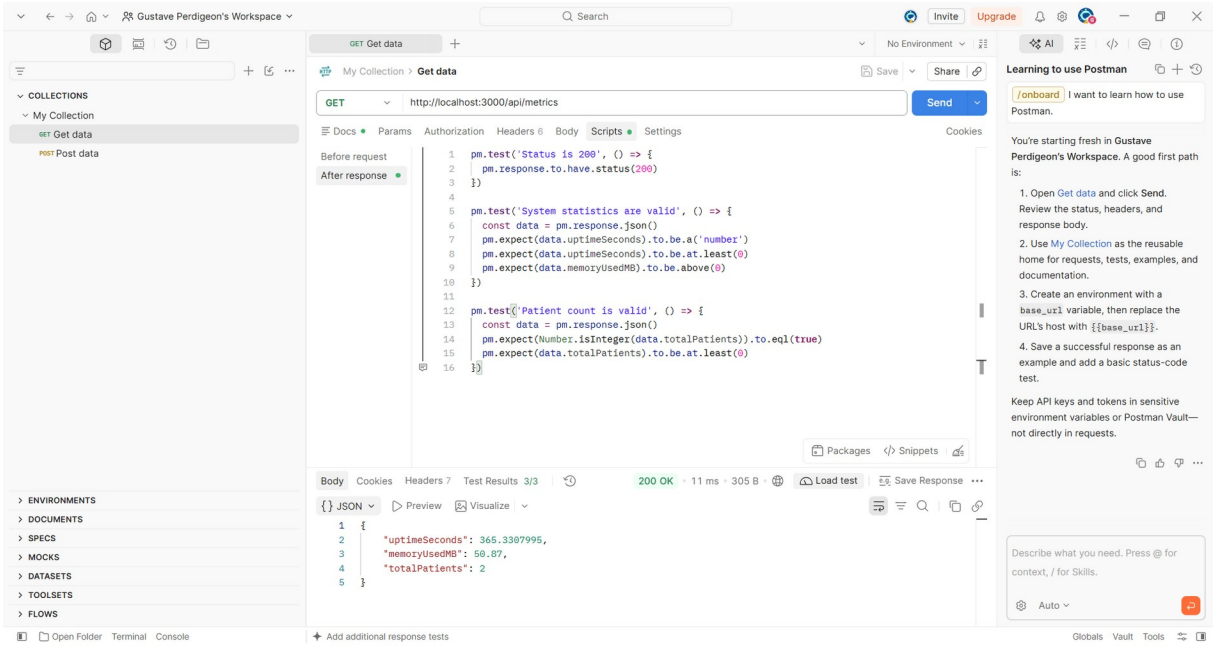
Zixuan Qian: GET /api/appointments, assertions passed.



Zixuan Qian: valid POST, assertions passed.

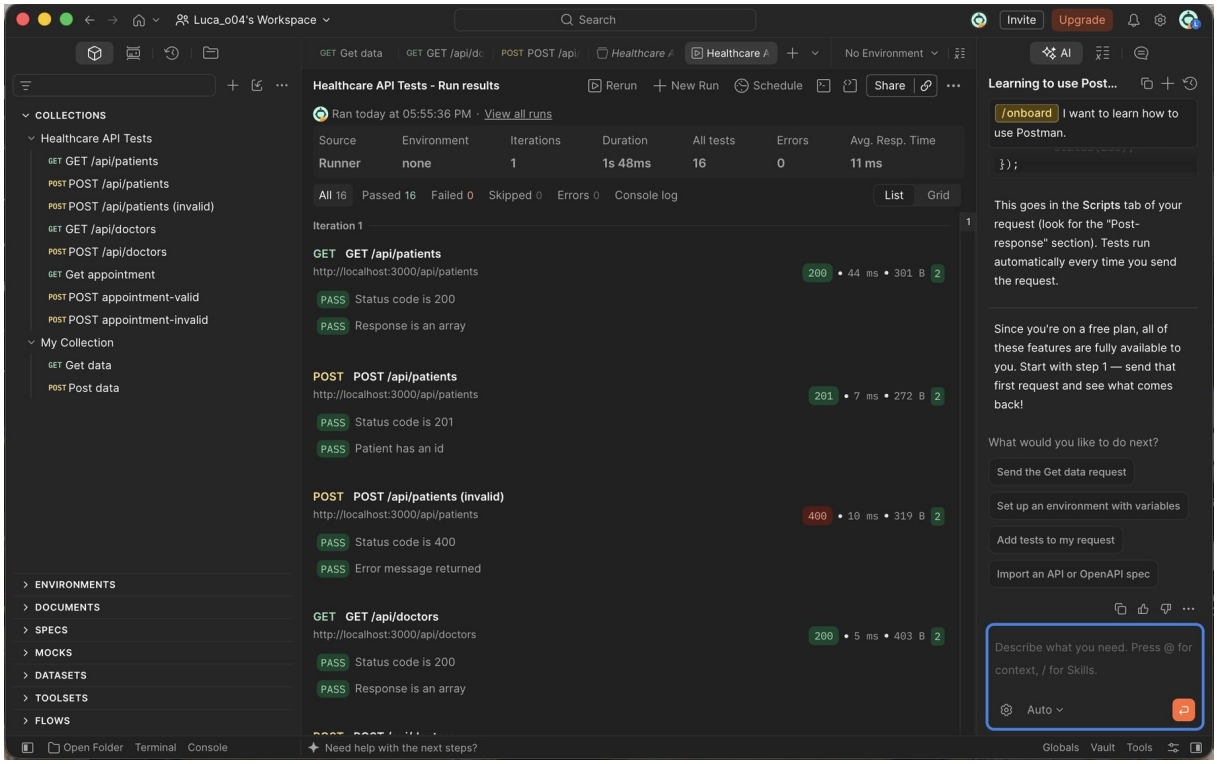


Zixuan Qian: invalid POST, assertions passed.

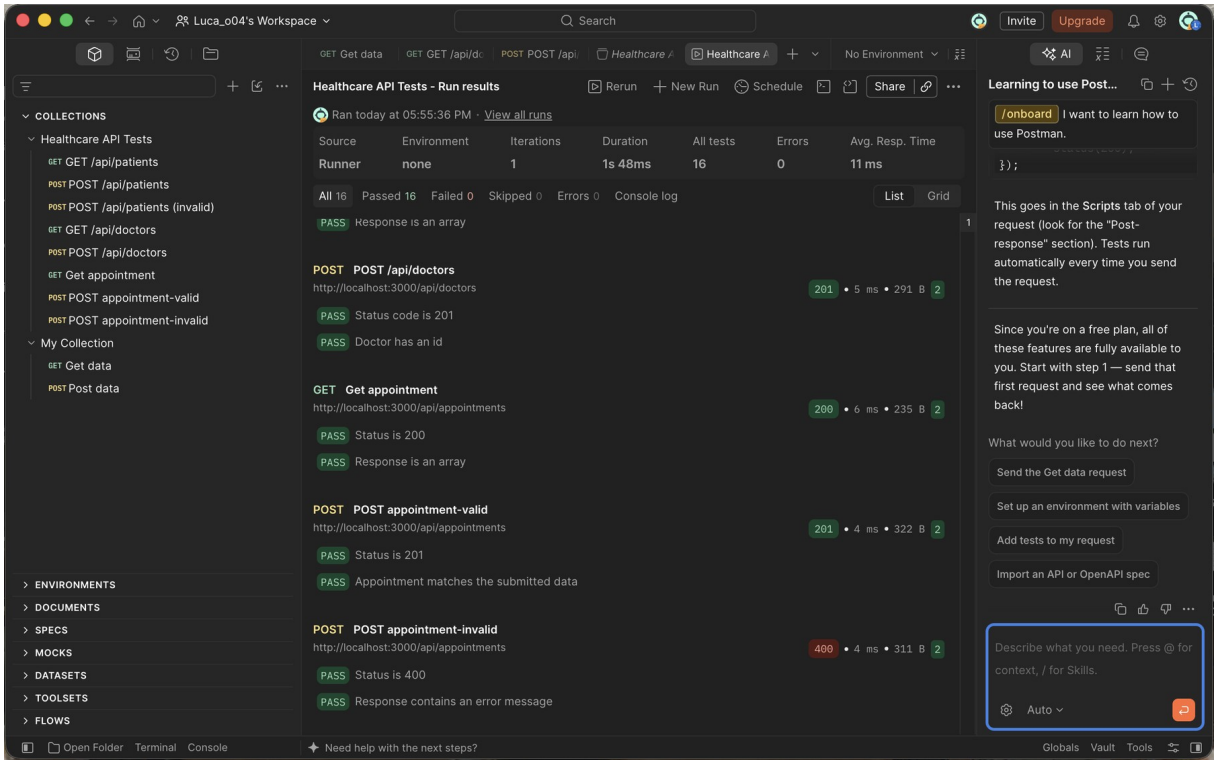


Gustave Perdigeon: GET /api/metrics, the three assertions passed.

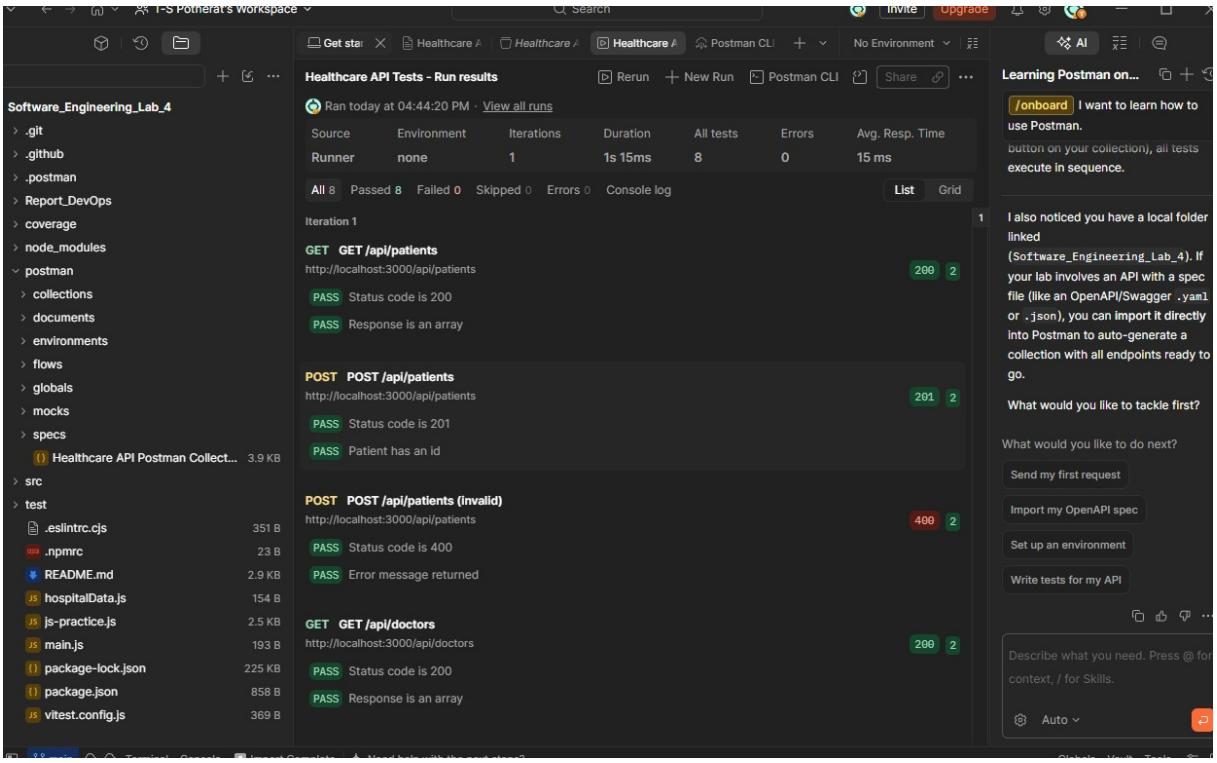
Running the collection



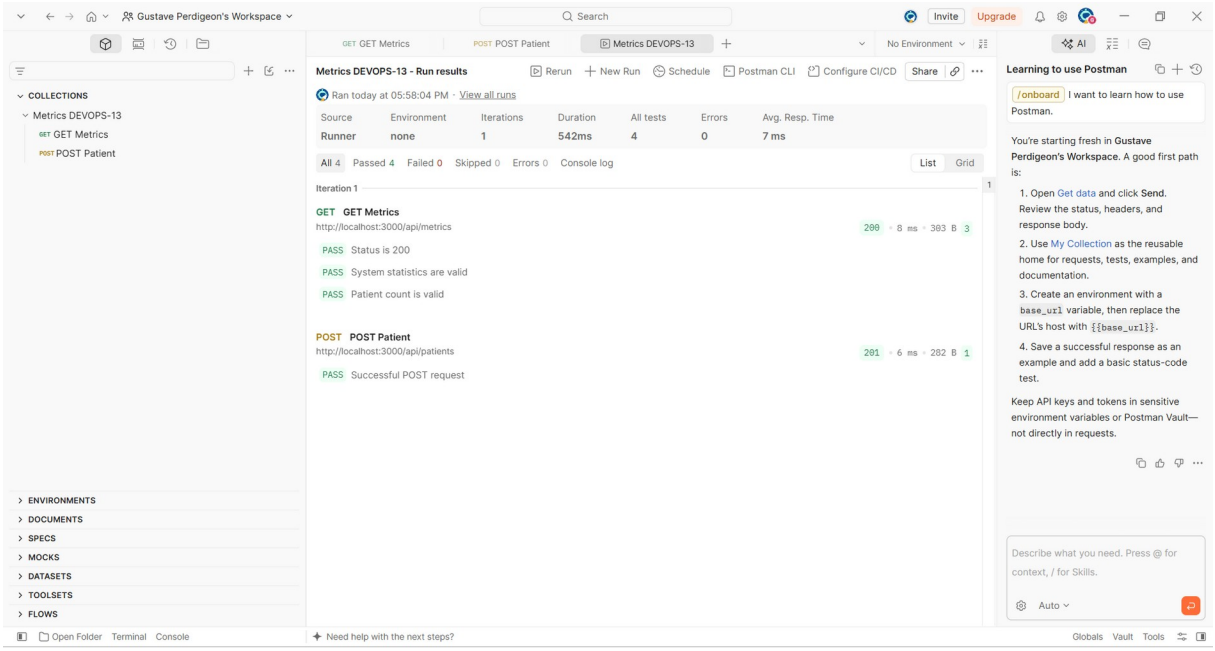
Luca Perkins: collection runner, 16 passed and 0 failed.



Luca Perkins: collection runner, rest of the results.



Thanh Potherat: collection runner, 8 assertions passed and 0 failed.



Gustave Perdigeon: collection runner, 4 assertions passed and 0 failed.

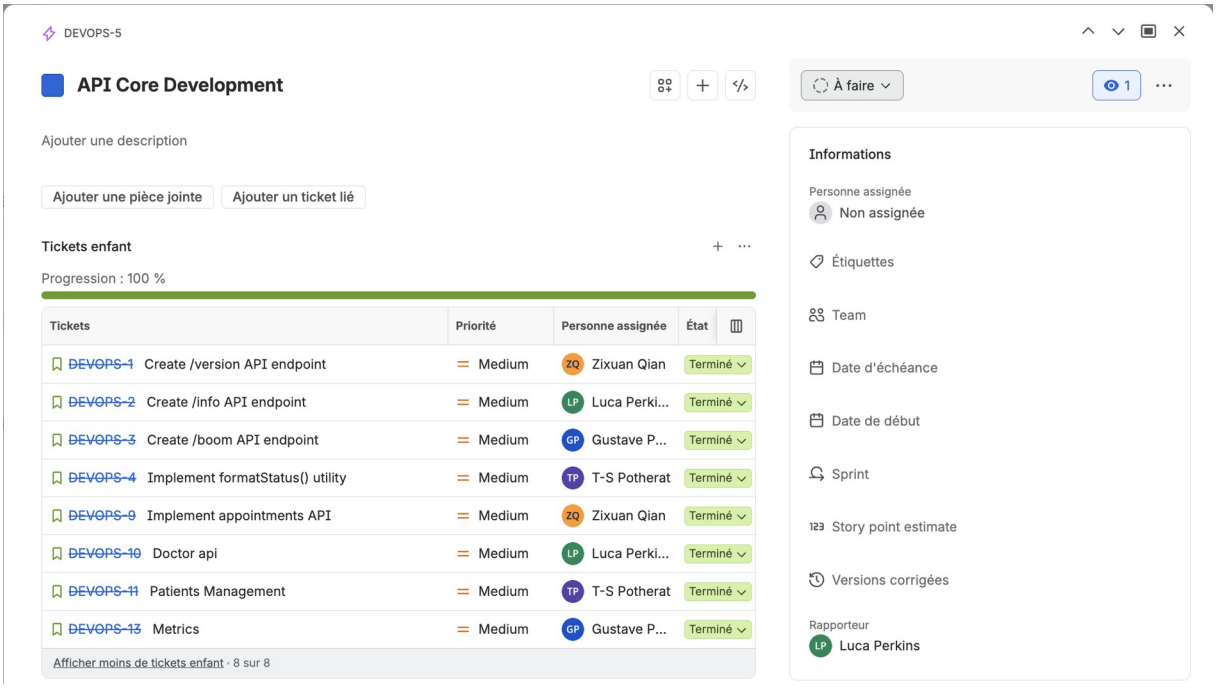
4.6 Export

The collections are exported in Postman v2.1 and committed under /postman/ in the repository.

5. DevOps workflow

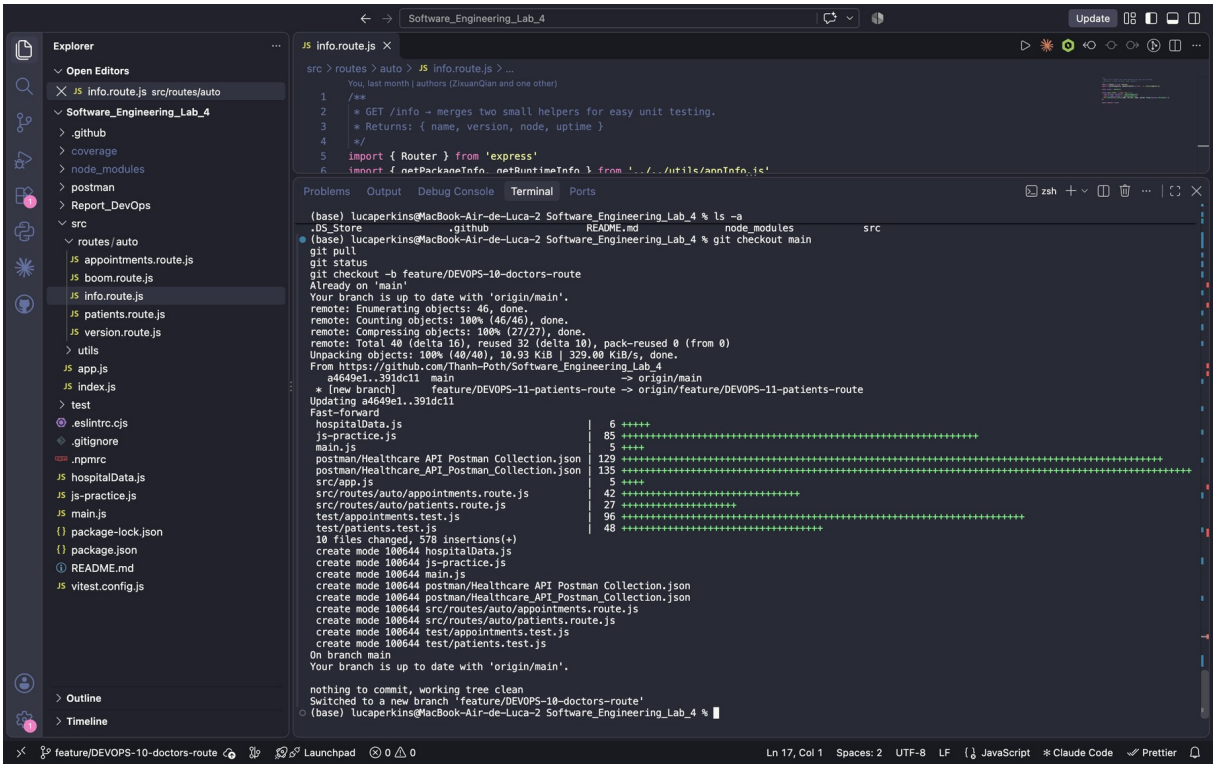
Each route followed the same steps: a Jira story, a branch, a commit, a push, then a pull request merged after the CI passed.

Jira epic and user stories

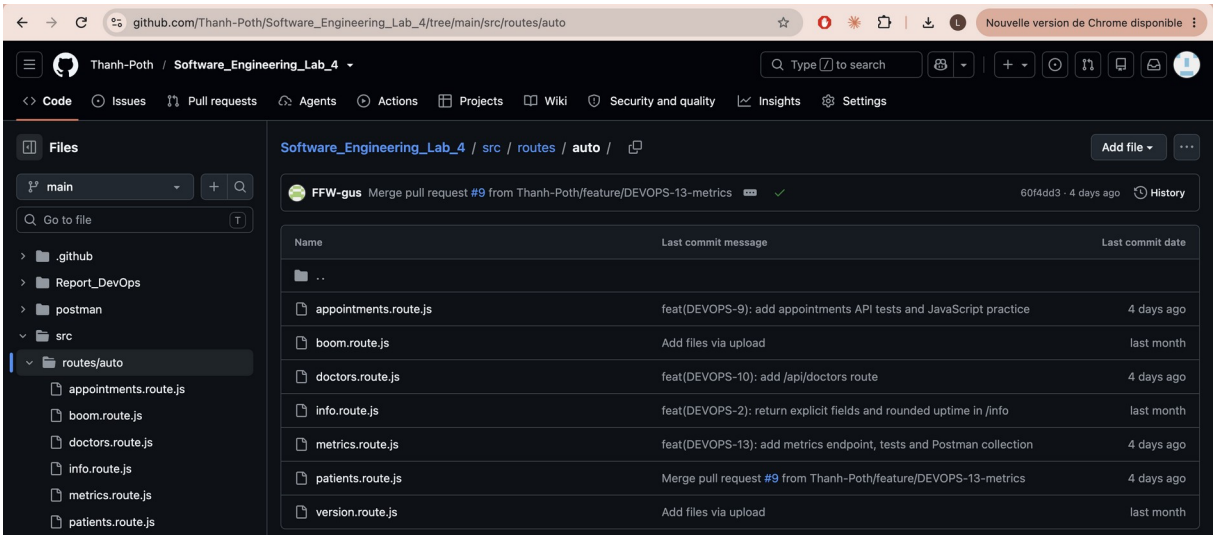


Epic DEVOPS-5 API Core Development with its child stories, all Done (Terminé).

5.1 Branch

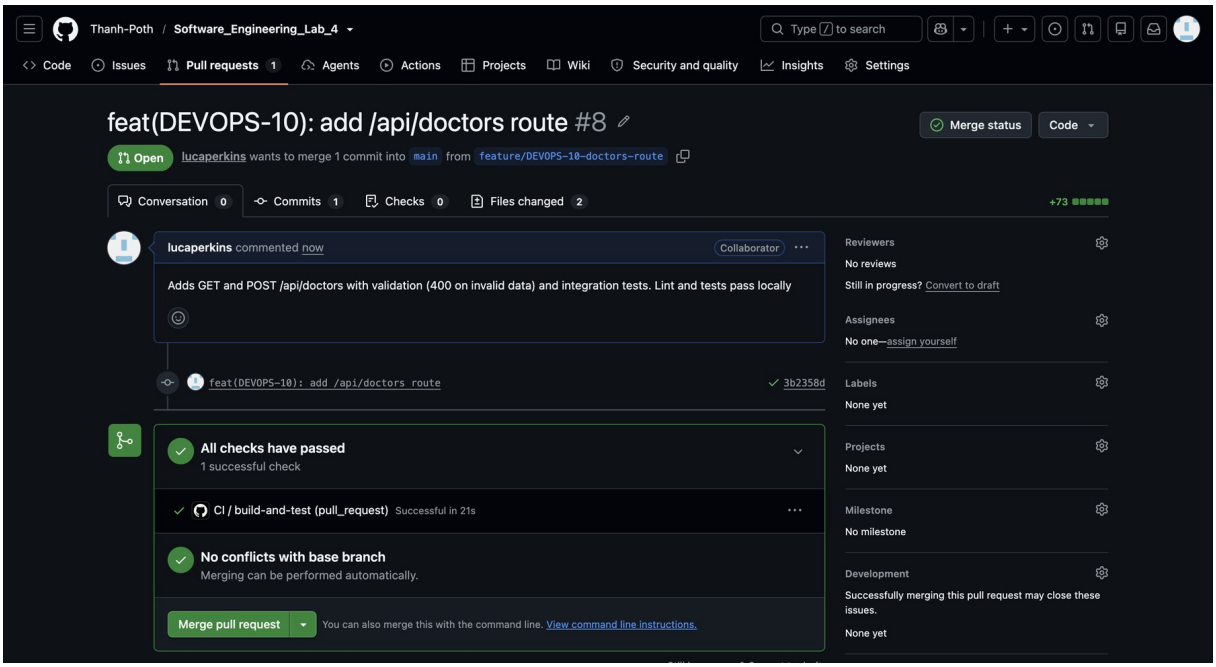

Software_Engineering_Lab_4
Update
Explorer
Open Editors
info.route.js src/routes/auto
Software_Engineering_Lab_4
.github
coverage
node_modules
postman
Report_DevOps
src
routes / auto
appointments.route.js
boom.route.js
info.route.js
patients.route.js
version.route.js
utils
app.js
index.js
test
.eslintrc.cjs
.gitignore
.npmrc
hospitalData.js
js-practice.js
main.js
package-lock.json
package.json
README.md
vitest.config.js
Outline
Timeline
src > routes > auto > info.route.js > ...
You, last month | authors (ZixuanQian and one other)
/**
* GET /info → merges two small helpers for easy unit testing.
* Returns: { name, version, node, uptime }
*/
import { Router } from 'express'
import { getPackageInfo, getRuntimeInfo } from '../../utils/appInfo.js'
Problems
Output
Debug Console
Terminal
Ports
zsh
(base) lucaperkins@MacBook-Air-de-Luca-2 Software_Engineering_Lab_4 % ls -a
.DS_Store .github README.md node_modules src
(base) lucaperkins@MacBook-Air-de-Luca-2 Software_Engineering_Lab_4 % git checkout main
git pull
git status
git checkout -b feature/DEVOPS-10-doctors-route
Already on 'main'
Your branch is up to date with 'origin/main'.
remote: Enumerating objects: 46, done.
remote: Counting objects: 100% (46/46), done.
remote: Compressing objects: 100% (27/27), done.
remote: Total 40 (delta 16), reused 32 (delta 10), pack-reused 0 (from 0)
Unpacking objects: 100% (40/40), 10.93 KiB | 329.00 KiB/s, done.
From https://github.com/Thanh-Poth/Software_Engineering_Lab_4
a4649e1..391dc11 main -> origin/main
* [new branch] feature/DEVOPS-11-patients-route -> origin/feature/DEVOPS-11-patients-route
Updating a4649e1..391dc11
Fast-forward
hospitalData.js | 6 +++++
js-practice.js | 85 ++
main.js | 5 ++++
postman/Healthcare API Postman Collection.json | 129 ++++++++++++++++++++++++++++++++++++++

Routes on GitHub

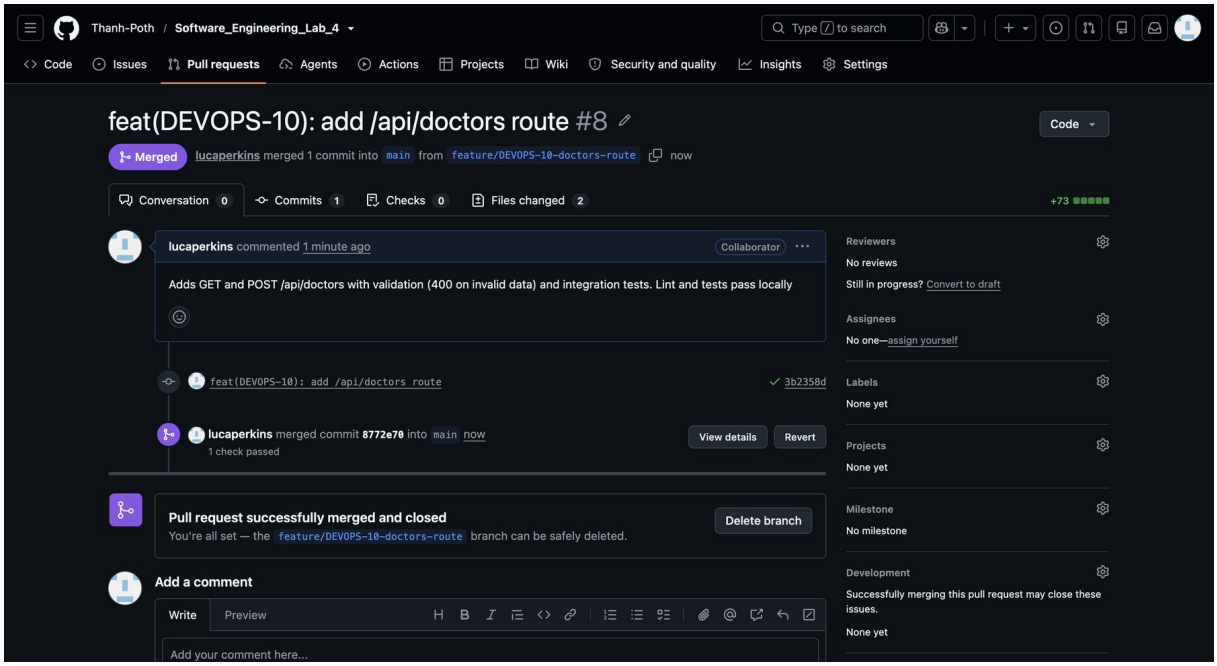


Folder src/routes/auto on main, with appointments, doctors, metrics and patients routes.

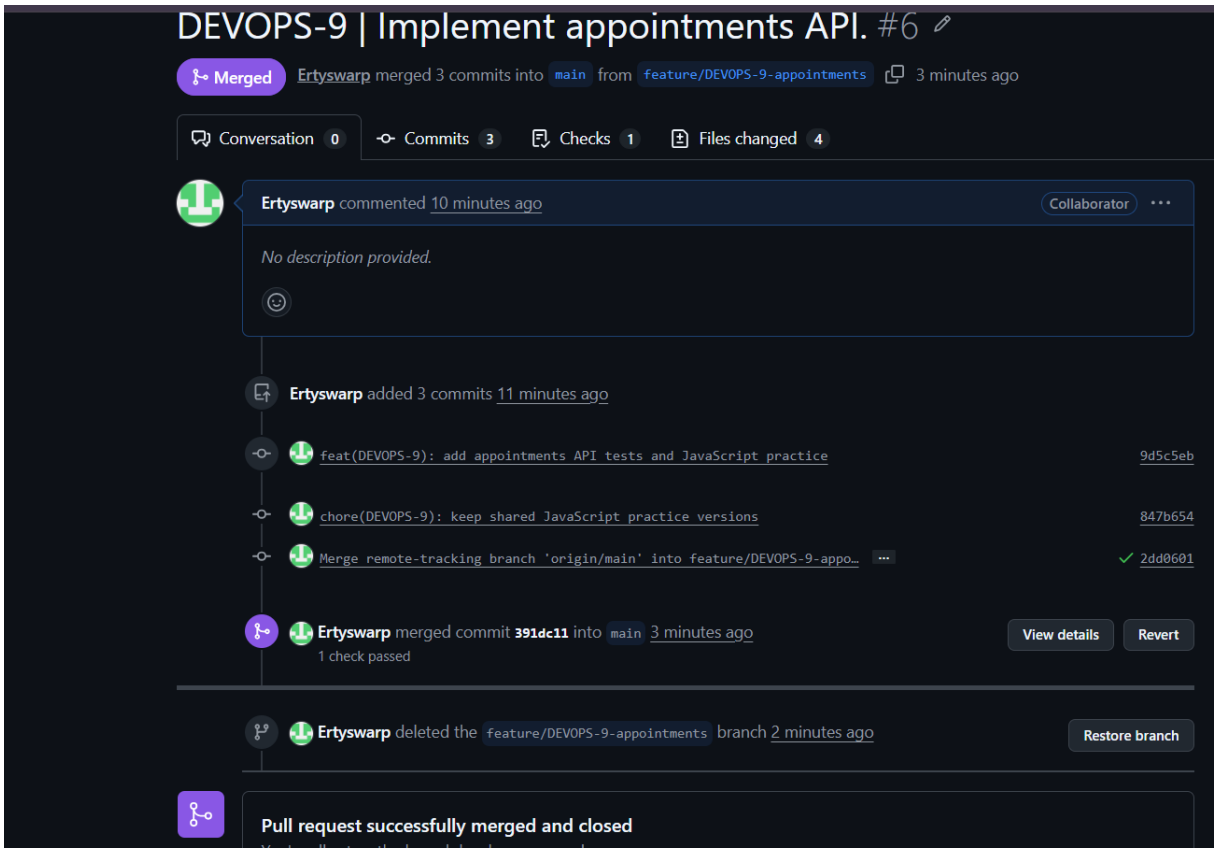
5.4 Pull request



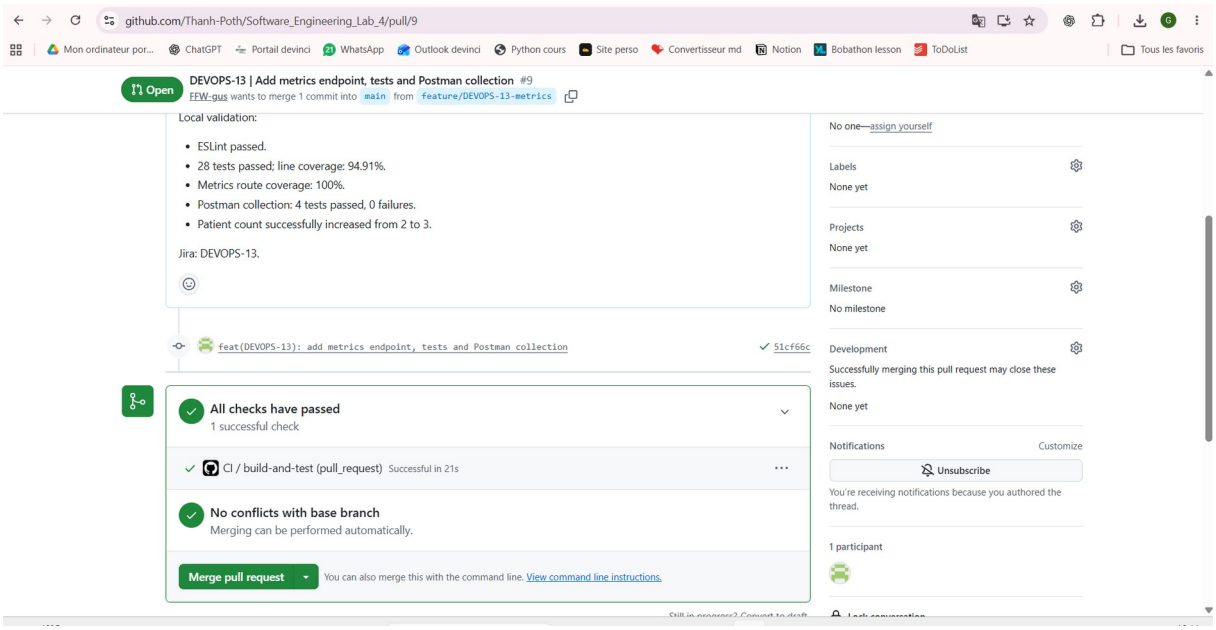
Luca Perkins: pull request #8, all checks passed.



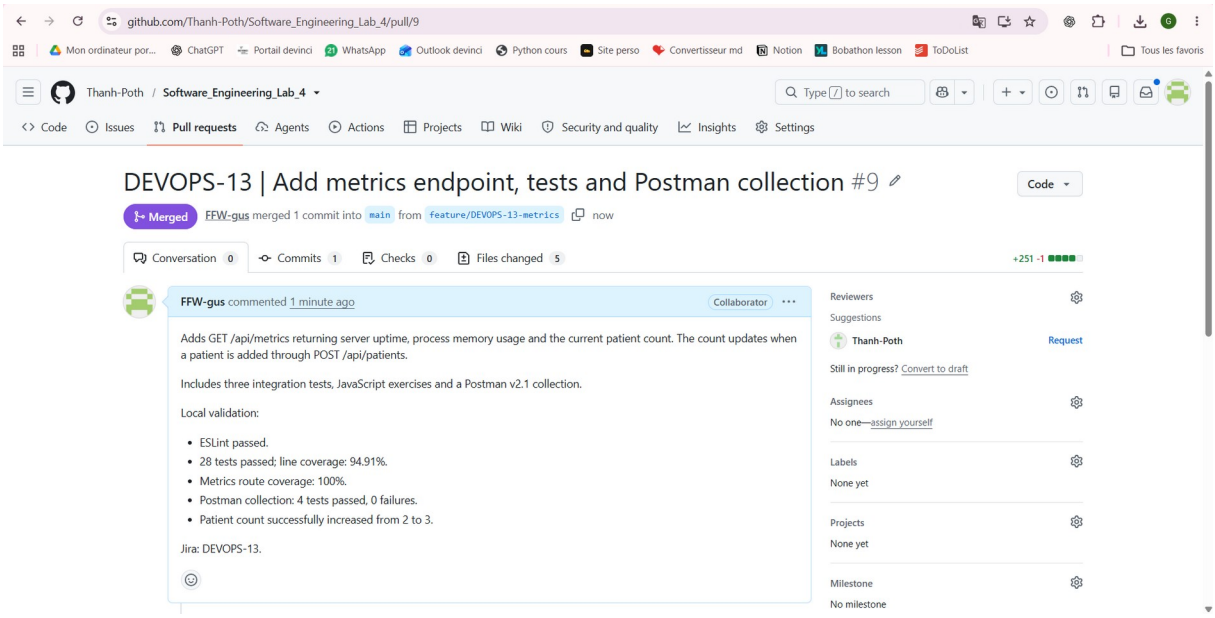
Luca Perkins: pull request #8 merged into main.



Zixuan Qian: pull request #6 merged into main.

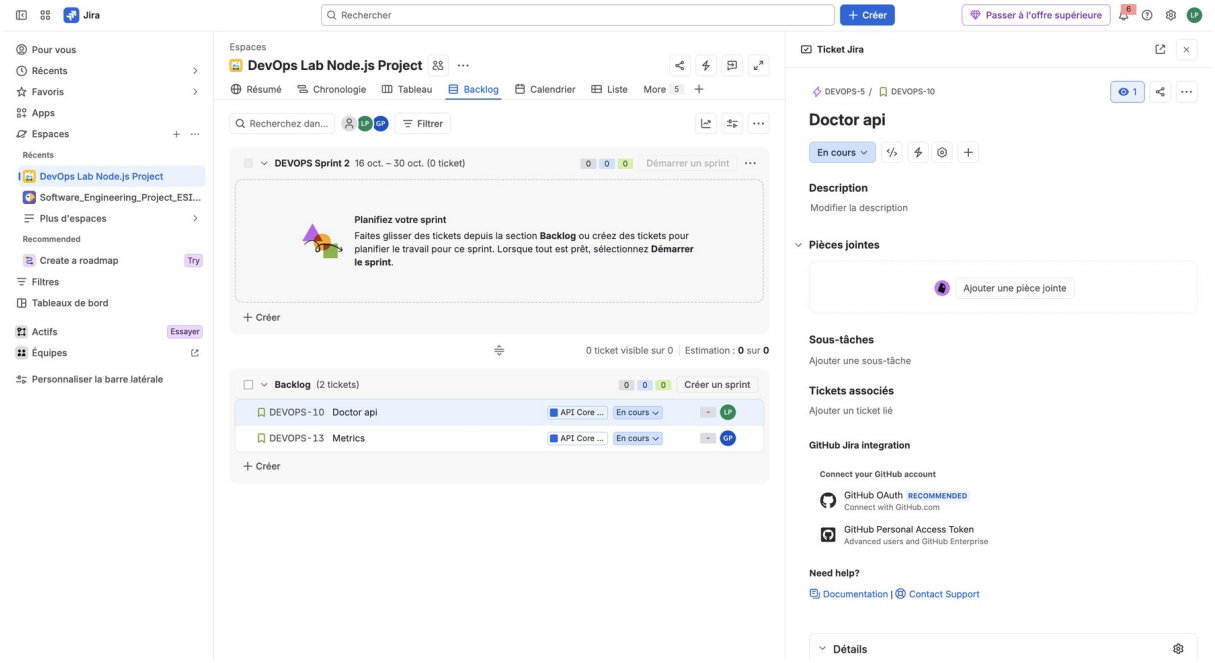


Gustave Perdigeon: pull request #9, CI check passed and no conflict.

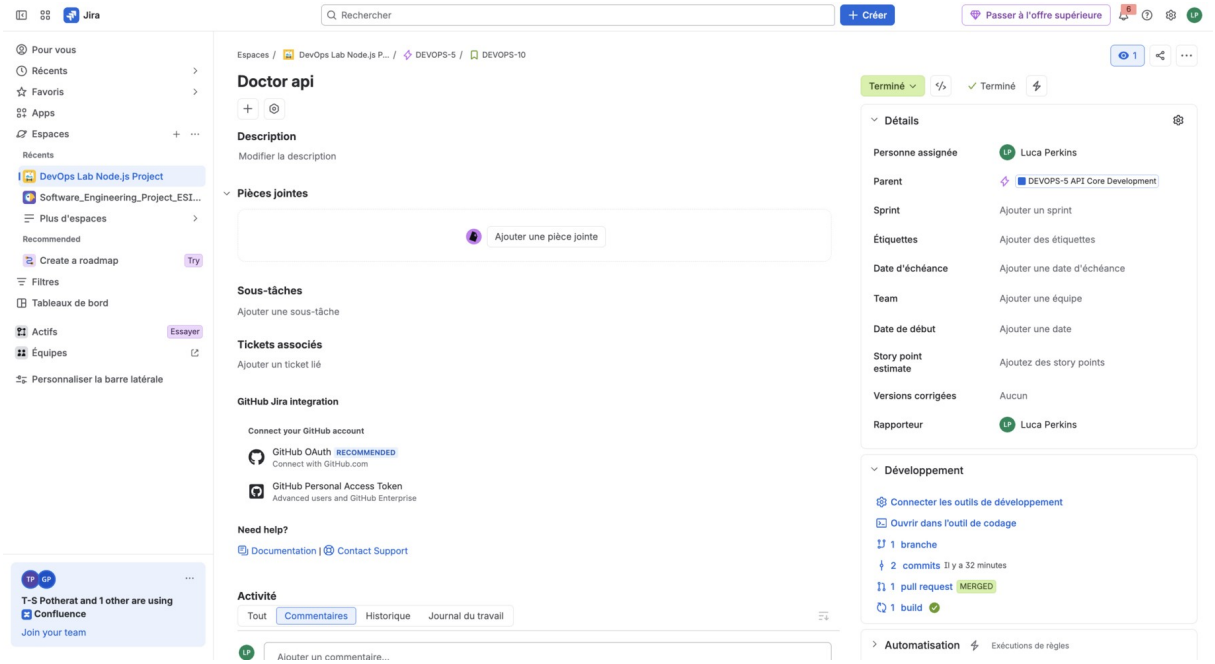


Gustave Perdigeon: pull request #9 merged into main.

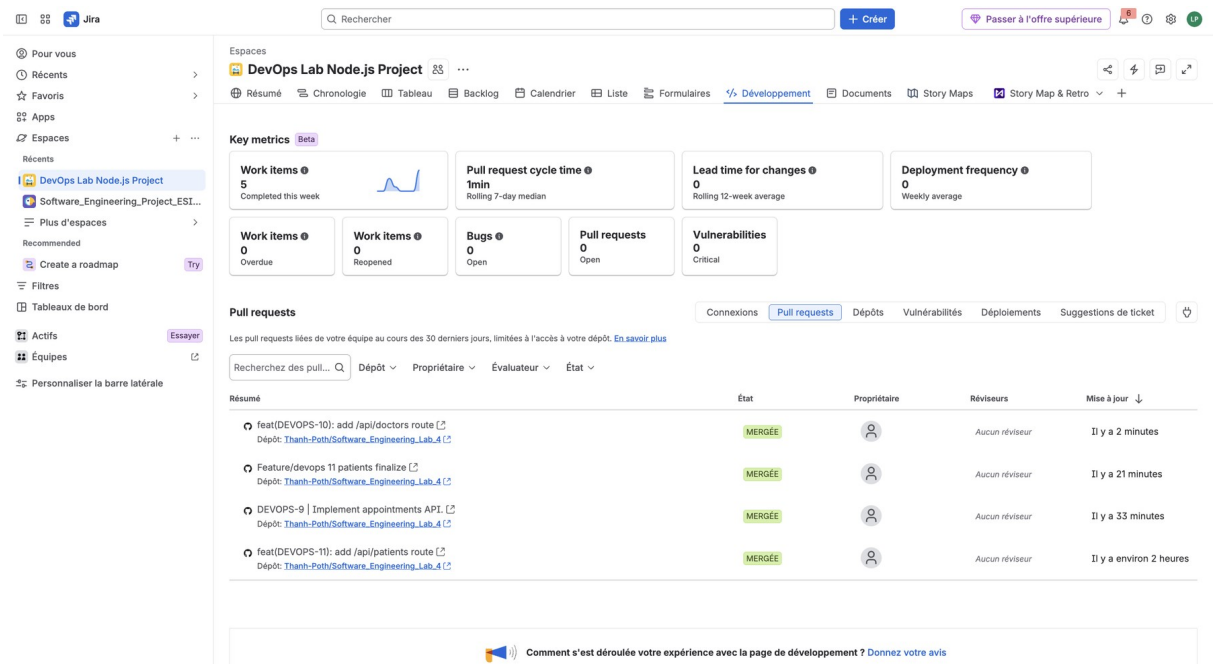
Jira story linked to the pull request



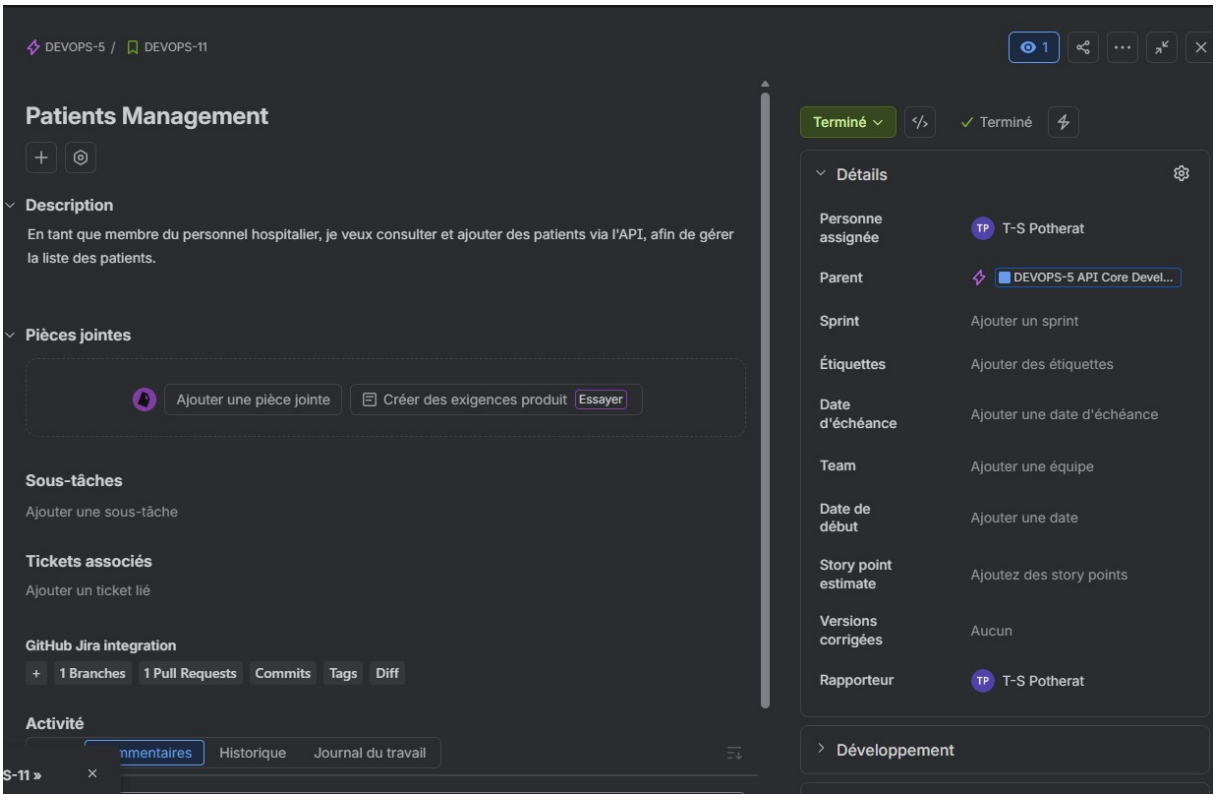
Luca Perkins: DEVOPS-10 Doctor api set to In Progress (En cours) on the board.



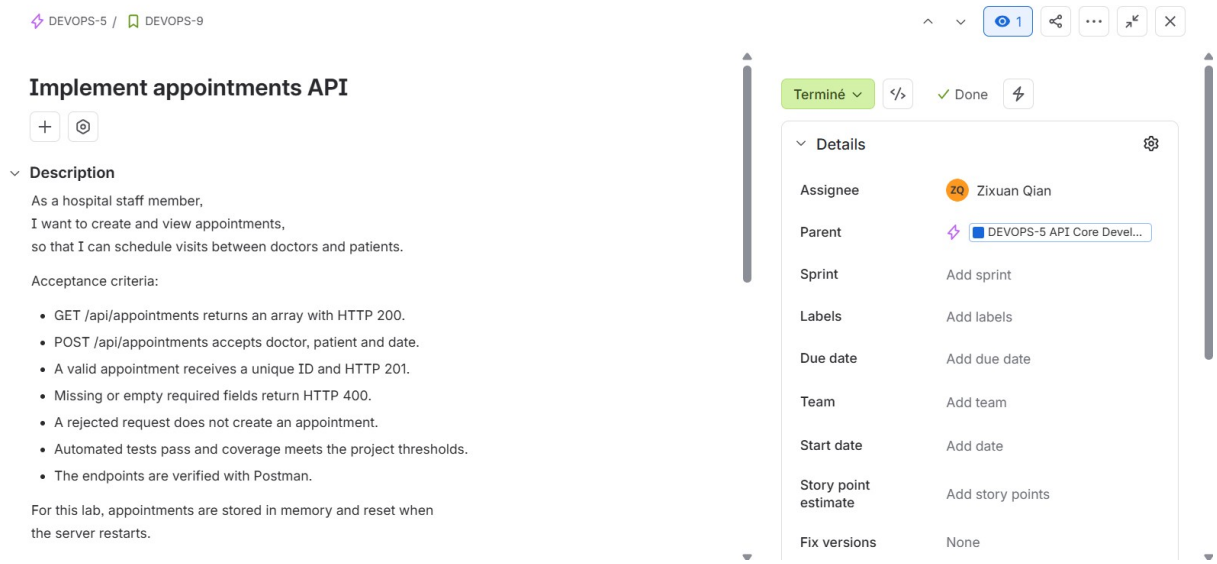
Luca Perkins: DEVOPS-10 is Done (Terminé), with 1 branch, 2 commits, 1 merged pull request and 1 green build.



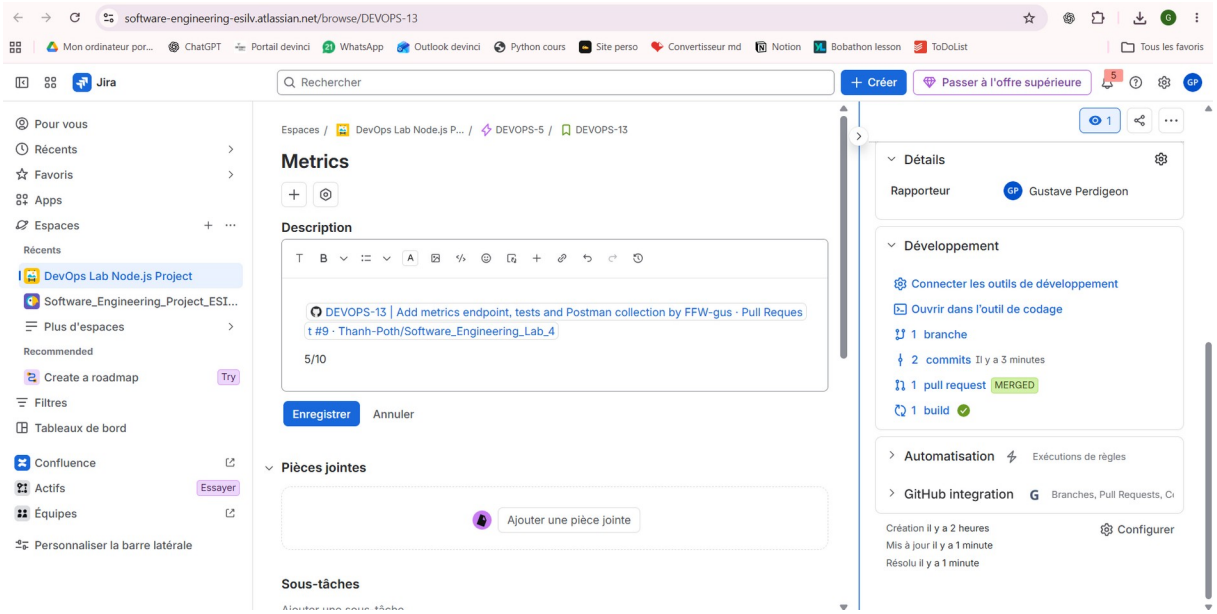
Luca Perkins: Jira development view listing the merged pull requests.



Thanh Potherat: DEVOPS-11 Patients Management is Done, linked to a branch and a pull request.



Zixuan Qian: DEVOPS-9 Implement appointments API is Done.

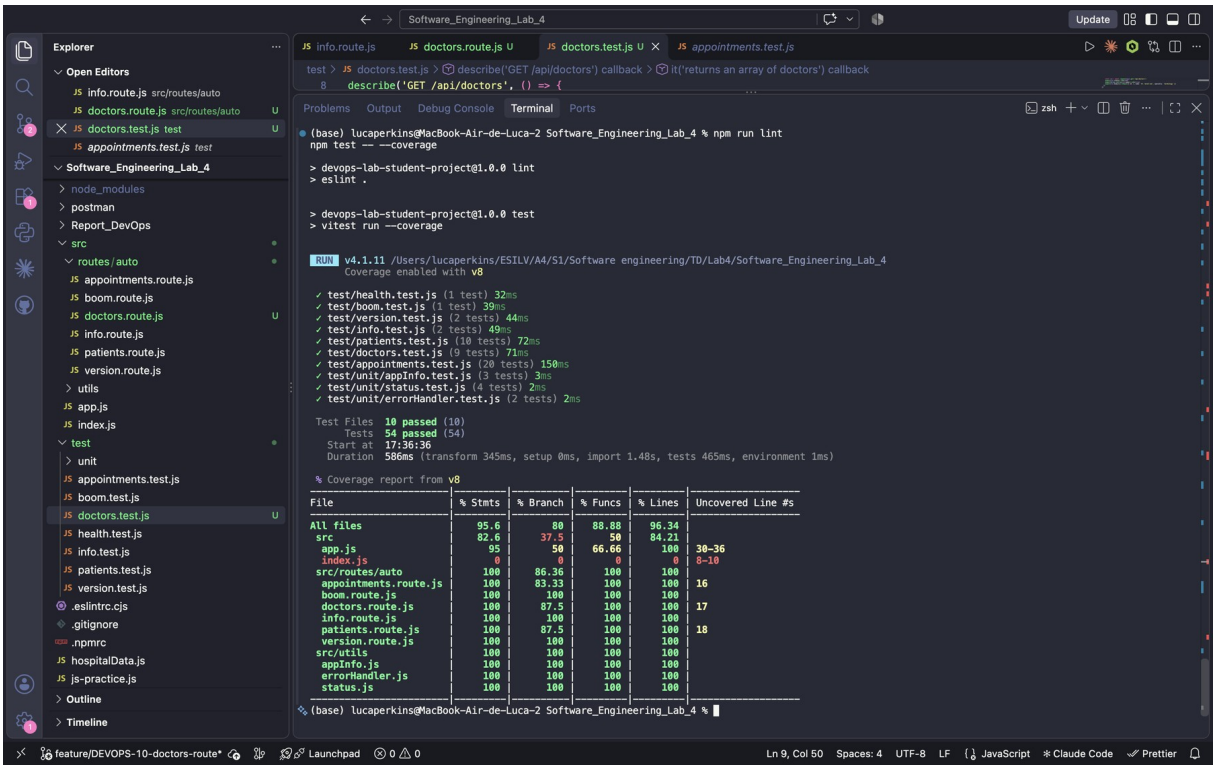


Gustave Perdigeon: DEVOPS-13 Metrics linked to the merged pull request and the green build.

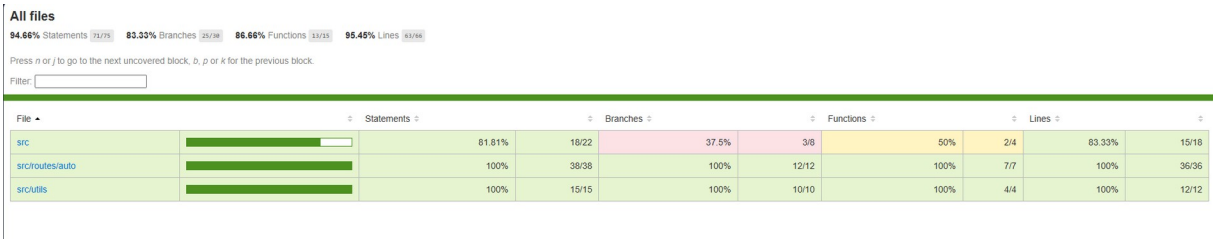
6. Continuous integration

The CI runs npm ci, npm run lint and npm test with coverage on every push and pull request. New routes must be tested and coverage must stay above the thresholds.

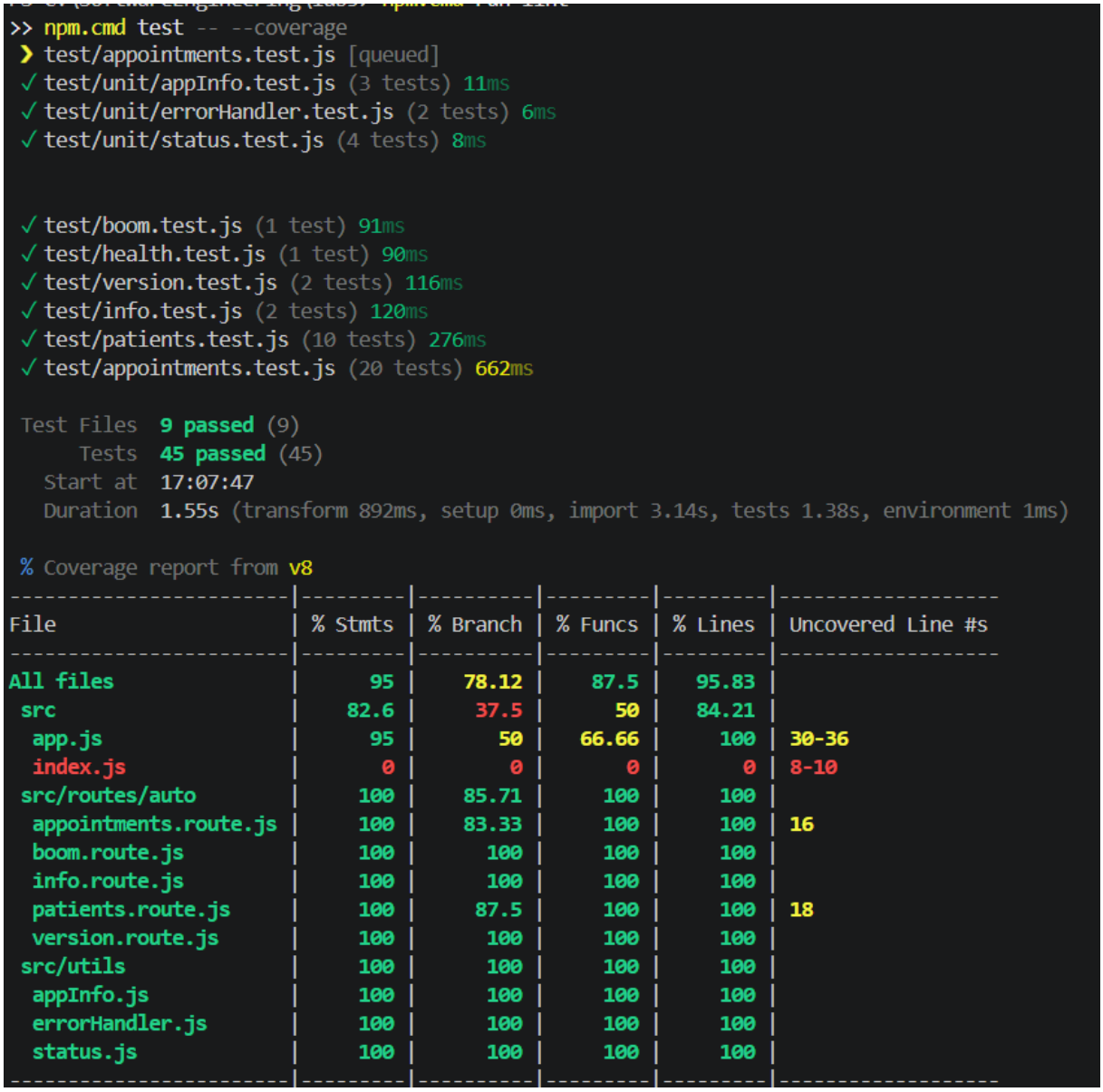
Lint, tests and coverage



Luca Perkins: npm run lint, then 54 tests passed in 10 files, with 95.6% of statements covered.



Thanh Potherat: coverage report, 94.66% of statements and 83.33% of branches.



Zixuan Qian: 45 tests passed, with the coverage table.

```
 RUN  v4.1.11 C:/Users/gusta/Documents/Software engineering/lab4/Software_Engineering_Lab_4
      Coverage enabled with v8

 ✓ test/unit/appInfo.test.js (3 tests) 15ms
 ✓ test/unit/errorHandler.test.js (2 tests) 9ms
 ✓ test/unit/status.test.js (4 tests) 9ms
 ✓ test/boom.test.js (1 test) 44ms
 ✓ test/info.test.js (2 tests) 62ms
 ✓ test/version.test.js (2 tests) 52ms
 ✓ test/health.test.js (1 test) 61ms
 ✓ test/metrics.test.js (3 tests) 110ms
 ✓ test/patients.test.js (10 tests) 154ms

 Test Files  9 passed (9)
      Tests  28 passed (28)
   Start at  17:47:01
   Duration  1.25s (transform 480ms, setup 0ms, import 3.39s, tests 516ms, environment 2ms)

 % Coverage report from v8
-------------------|---------|----------|---------|---------|-------------------
File               | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-------------------|---------|----------|---------|---------|-------------------
All files          |   94.02 |    76.92 |   85.71 |   94.91 |
 src               |   81.81 |     37.5 |      50 |   83.33 |
  app.js           |   94.73 |       50 |   66.66 |     100 | 29-35
  index.js         |       0 |        0 |       0 |       0 | 8-10
 src/routes/auto   |     100 |     87.5 |     100 |     100 |
  boom.route.js    |     100 |      100 |     100 |     100 |
  info.route.js    |     100 |      100 |     100 |     100 |
  metrics.route.js |     100 |      100 |     100 |     100 |
  patients.route.js|     100 |     87.5 |     100 |     100 | 18
  version.route.js |     100 |      100 |     100 |     100 |
 src/utils         |     100 |      100 |     100 |     100 |
  appInfo.js       |     100 |      100 |     100 |     100 |
  errorHandler.js  |     100 |      100 |     100 |     100 |
  status.js        |     100 |      100 |     100 |     100 |
-------------------|---------|----------|---------|---------|-------------------
```

Gustave Perdigeon: 28 tests passed, the metrics route is covered at 100%.

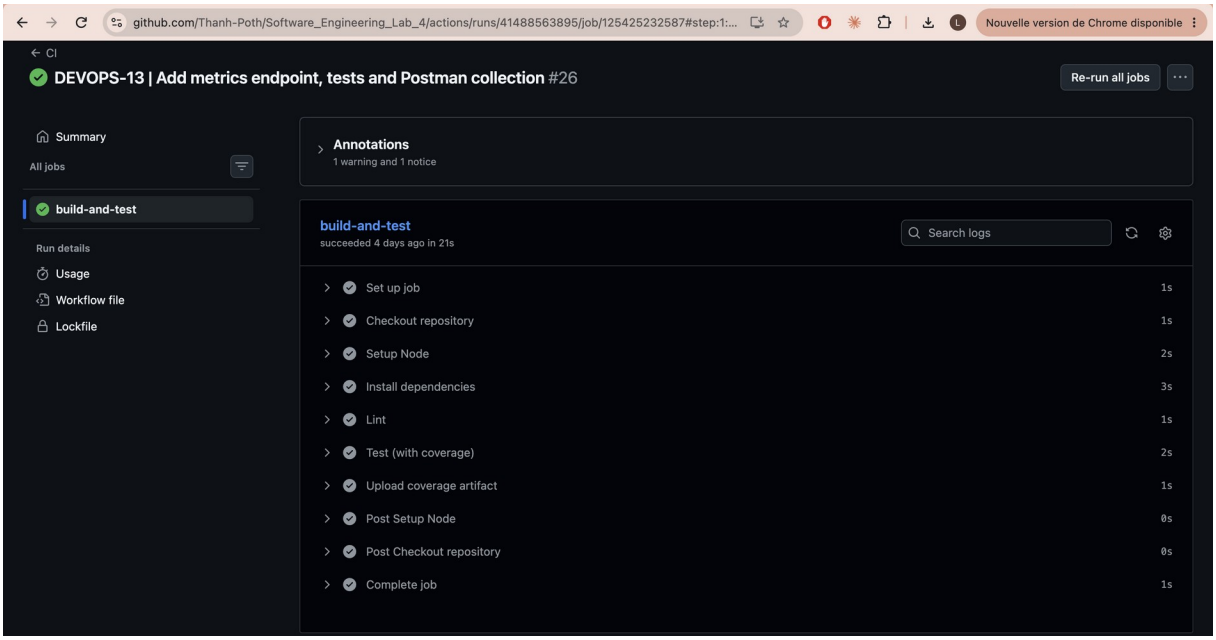
```
PS C:\Users\gusta\Documents\Software engineering\lab4\Software_Engineering_Lab_4> npm.cmd run lint

> devops-lab-student-project@1.0.0 lint
> eslint .

PS C:\Users\gusta\Documents\Software engineering\lab4\Software_Engineering_Lab_4>
```

Gustave Perdigeon: npm run lint finishes without errors.

GitHub Actions



CI run #26: the build-and-test job is green, with its steps (checkout, setup Node, install, lint, tests with coverage, upload).



Thanh Potherat: CI runs on pull request #5 and on main, all green.